密级:_____

中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

龙芯 MIPS 64 位 Java 虚拟机的实现与优化

作者姓名：武　慧

指导教师：张戈　副研究员

中国科学院计算技术研究所

学位类别：工程硕士

学科专业：计算机技术

研 究 所：中国科学院计算技术研究所

2013 年　5 月

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

Implementation and Optimization of Loongson 64-bit Java Virtual Machine

By

Wu Hui

A Dissertation Submitted to

University of Chinese Academy of Sciences

In partial fulfillment of the requirement

For the degree of

Master of Engineering

Institute of Computing Technology, Chinese Academy of Sciences

May, 2013

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

摘 要

随着国内外处理器技术的飞速发展，简洁高效的 MIPS 指令集系统已经广泛应用于服务器、高性能计算等领域。相比与 X86 平台，MIPS 平台上应用程序不够丰富已成为制约其发展的主要问题。为此，应用程序跨平台支持就显得尤为重要。Java 由于其良好的跨平台特性，已经成为跨平台实现的重要途径。Java 程序的运行需要 Java 虚拟机的支持。现已实现的仅支持 MIPS32 指令集的 Java 虚拟机由于内存堆栈分配受限、并发吞吐量较小等局限，已无法满足服务器领域高并发、高吞吐的需求。为此，设计和实现一个稳定高效的支持 MIPS64 指令集的 Java 虚拟机，克服原虚拟机中 32 位指令集的各项固有限制，已变得非常迫切。

本文完成的工作和主要贡献如下：

第一，以 OpenJDK 6 为基础，实现了基于龙芯 MIPS 64 平台的 Java 虚拟机系统。本文分析和研究了 Java 虚拟机的结构和模块间的交互关系，实现了基于 MIPS64 指令集的模板化解释器和更高效率的即时编译器，解决了 Java 虚拟机实现的关键环节问题，包括字节码的解析和执行、方法调用、传参实现、异常处理、JNI 接口调用等。

第二，采用查找表的方式，对获取 Java 线程控制结构首地址方法进行优化，提高了 Java 线程控制结构首地址的获取效率。优化后，通过 SPECjvm2008 测试表明，性能平均提升 13.7%。在前一项优化的基础上采用寄存器缓冲的方式进一步优化，性能最高提升 12.9%。

第三，采用从编译执行调用 native 方法的机制，对 JNI 接口调用进行优化，提高了 native 方法的编译执行效率。优化后测试 SPECjvm2008 中的程序，最高获得了 59.5%的性能提升，平均性能提升 9.1%。

第四，采用低 32 位地址空间分配的方式，对 Safepoint 页面地址分配进行优化，提升了 Safepoint 页面地址的访问效率，优化后测试 SPECjvm2008 中的程序，性能最高提升 11.8%。

本文完成的 MIPS 64 位 Java 虚拟机在 3GB 内存下，比 32 位 Java 虚拟机平均性能提升 18.97%。在通过多项稳定性和压力测试之后，现已被集成到龙芯的实际应用产品中。

关键词：OpenJDK 6，JIT 编译器，MIPS 64，Java 虚拟机，JNI 接口

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

ABSTRACT

With the rapid development of processor technology, concise and efficient MIPS instruction set system has been used in the server and high performance computing field. Compared with the X86 platform，lacking of applications on the MIPS 64-bit platform has become the major issue of restrcting its development. Therefore, cross-platform supporting of the applications has become very important. Java has become an important way to achieve cross-platform because of its good cross-platform features. The running of Java programs needs the supportting of the Java Virtual Machine. Now The Java virtual machine realized only supports MIPS 32-bit instruction set, the memory space it can use is limited and its concurrent throughput is smaller. Because of those limitations, it can not meet the needs of server field of high concurrency, high throughput. So, design and implement a stable and efficient Java virtual machine which support MIPS 64-bit instruction set to overcome the existing inherent limitations of 32-bit instruction set has become very urgent.

The completed work and main contributions of this article are as follows:

1. Completed the transplantation of MIPS 64-bit instruction set of Loongson Java virtual machine. We analyze the structure of OpenJDK 6 and the interactive relationship of each module. We complete the transplantation of template interpreter based on the MIPS 64-bit instruction set and the transplantation of higher efficiency just-in-time compilers. We have also resolved the main issues of Java virtual machine transplantation, including analysis of the bytecodes, method invocation, parameter passing, exception handling, JNI interface calls, etc.

2. We use a lookup table to optimize the method of getting the first address of Java thread control structure, improving the efficiency of getting the address pointer. After the optimization, the corresponding evaluation shows that it improves the performance of SPECjvm2008 by 13.7% on average. Then, we use a callee-saved register to cache the thread pointer. The evaluation of the results shows that the highest performance increased by 12.9% after optimized.

3. We implement a mechanism of calling native methods from compiling execution, improved the compiling execution efficiency of the native method. After optimization, the highest performance of SPECjvm2008 improved by 59.5%,

average improved by 9.1%.

4. We use the method of low 32-bit address space allocation promoted the efficiency of safepoint page address accessing. The evaluation of the results shows that the highest performance of SPECjvm2008 increased by 11.8%.

In the conditions of 3G memory, Compared with the 32-bit Java virtual machine, The corresponding evaluation shows that the 64-bit Java virtual machine which completed in this article improves the performance of SPECjvm2008 by 18.97% on average. After passing a number of stability and stress test, it has been integrated into the Loongson products.

Key Words: OpenJDK 6, JIT Compiler, MIPS 64, Java Virtual Machine, Java Native Interface

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

目 录

摘 要 ..... **I**
ABSTRACT ..... **III**
目 录 ..... **V**
图目录 ..... **IX**
表目录 ..... **XI**
第一章 绪论 **1**
1.1 MIPS 64 位龙芯平台简介 1
1.2 服务器应用程序 3
1.3 Java 平台介绍 3
1.3.1 Java 程序设计语言特点 4
1.3.2 Java 虚拟机核心技术 4
1.3.3 Just-in-time 编译 6
1.4 MIPS 64 位 Java 虚拟机移植的必要性 8
1.5 现有主流 Java 虚拟机 9
1.6 本文研究内容与研究工作 10
1.7 本文的组织结构 11
第二章 MIPS 64 位 Java 虚拟机的实现 **13**
2.1 OpenJDK 6 结构 13
2.2 实现模块相关工作分析 15
2.3 MIPS 64 位虚拟机寄存器分配实现 17
2.3.1 MIPS 64 位寄存器规范 17
2.3.2 寄存器分配策略实现 18
2.4 模板解释器的 MIPS 64 位实现 20
2.4.1 模板生成函数实现 21
2.4.2 模板解释生成器实现 24
2.5 JIT 编译器的 MIPS 64 位实现 27
2.5.1 编译栈帧映射实现 27
2.5.2 LIR 生成器实现 29
2.5.3 LIR 汇编器实现 29

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

2.6 运行时环境实现 32
2.6.1 运行时方法调用传参机制 32
2.6.2 适配器实现 34
2.7 调试手段和问题解决实例 34
2.7.1 随机多线程错误案例 35
2.7.2 低频率错误案例 37
2.8 本章总结 40
第三章 龙芯 MIPS 64 位 Java 虚拟机优化 41
3.1 获取 Java 线程控制结构首地址的优化方法 41
3.1.1 Java 线程控制结构首地址实现 42
3.1.2 获取 Java 线程控制结构首地址方法频繁性验证 43
3.1.3 获取 Java 线程控制结构首地址-查找表优化 43
3.1.4 获取 Java 线程控制结构首地址-寄存器缓存优化 44
3.1.5 获取 Java 线程控制结构首地址优化效果 46
3.2 JNI 接口调用优化 46
3.2.1 Native 方法的调用机制 46
3.2.2 编译执行调用 native 方法优化实现 47
3.2.3 JNI 接口调用优化效果 49
3.3 Safepoint 优化 50
3.3.1 Safepoint 实现机制 50
3.3.2 Safepoint 页面分配优化 51
3.3.3 Safepoint 页面分配优化效果 53
3.4 本章总结 53
第四章 MIPS 64 位 Java 虚拟机性能评测 55
4.1 测试平台及工具介绍 55
4.2 获取 Java 线程控制结构首地址优化效果 56
4.2.1 获取 Java 线程控制结构首地址方法调用次数统计 56
4.2.2 获取 Java 线程控制结构首地址-查找表优化效果 57
4.2.3 获取 Java 线程控制结构首地址-寄存器缓存优化效果 59
4.3 JNI 接口调用优化效果 60
4.4 Safepoint 页面分配优化效果 62
4.5 三项优化综合效果 64

4.6 64 位虚拟机与 32 位虚拟机性能对比与分析 66
4.7 本章总结 68
第五章 总结与展望 69
5.1 本文工作总结 69
5.2 下一步工作和研究方向 69
参考文献 71
致　谢 75
作者简介及在学期间发表的学术论文与研究成果 77

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

图目录

图 2- 1 HotSpot 结构图.....14
图 2- 2 模板解释器执行流程图.....20
图 2- 3 ldc 模板生成函数实现流程图.....22
图 2- 4 lmul 模板生成函数 32 位与 64 位实现对比图.....23
图 2- 5 解释栈帧结构图.....24
图 2- 6 32 位与 64 位参数传递实例图.....26
图 2- 7 JIT 编译器执行流程图.....27
图 2- 8 LIR 操作数位图.....28
图 2- 9 低级中间表示结构图.....29
图 2- 10 解释与编译转换适配器结构图.....34
图 2- 11 出错位置代码片段.....36
图 2- 12 错误日志片段.....38
图 3- 1 Java 线程运行时栈地址空间图.....43
图 3- 2 优化前、后不同执行模式下调用 native 方法对比图.....47
图 3- 3 本地方法调用栈结构图.....48
图 4- 1 获取 Java 线程控制结构首地址方法调用次数统计图.....56
图 4- 2 获取 Java 线程控制结构首地址方法-查找表优化效果图.....58
图 4- 3 获取 Java 线程控制结构首地址-寄存器缓存优化效果图.....59
图 4- 4 JNI 接口调用优化效果图.....61
图 4- 5 Safepoint 页面分配优化效果图.....63
图 4- 6 三项优化综合效果图.....65
图 4- 7 32 位与 64 位 Java 虚拟机性能对比图.....67

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

表目录

表 2- 1 MIPS32 与 MIPS64 整型寄存器约定对比表 17
表 2- 2 MIPS32 与 MIPS64 浮点寄存器约定对比表 18
表 2- 3 参数传递约定 o32 与 n64 对比表 25
表 2- 4 随机崩溃错误修改前、后测试结果对比表 37
表 2- 5 虚拟机实际运行指令与源码对应表 38
表 2- 6 多 CPU 环境下 checkcast 字节码读写线程执行对照表 39
表 2- 7 解释器崩溃错误修改前、后测试结果对比表 40
表 3- 1 32 位与 64 位 Java 虚拟机中 get_thread 方法的汇编指令对比表 41
表 3- 2 get_thread 方法查找表优化前、后代码片段对照表 44
表 3- 3 优化前、后虚拟机执行 get_thread 方法的汇编指令对比表 44
表 3- 4 JNI 接口调用优化前、优化后实现对比 49
表 3- 5 32 位与 64 位 Java 虚拟机执行 safepoint 的汇编指令对比表 51
表 3- 6 加载立即数优化前、后实现对比表 52
表 3- 7 Safepoint 页面分配优化前、后实现对比表 53
表 4- 1 SPECjvm2008 测试集功能表 55
表 4- 2 Java 线程控制结构首地址-查找表优化前、后测试数据对比表 57
表 4- 3 JNI 接口调用优化前、后测试数据对比表 60
表 4- 4 Safepoint 页面分配优化前、后测试数据对比表 62
表 4- 5 三项优化前、后测试数据对比表 64
表 4- 6 32 位与 64 位 Java 虚拟机 SPECjvm2008 测试数据对比表 66

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

第一章 绪论

近年来，伴随着集成电路设计与工艺技术水平的不断提高，MIPS 指令集兼容的高性能处理器已经广泛应用于低能耗数据中心、高端桌面应用、服务器、高性能计算机等领域。然而，目前绝大多数应用程序都是基于 X86 平台实现的。MIPS 平台上应用程序匮乏已经极大的限制了 MIPS 高性能处理器的推广和应用。为此，扩充和丰富现有 MIPS 软件库平台就变得非常急切和必要。这使得我们需要寻找一种快速扩充现有软件库的解决方法—应用程序跨平台支持。

Java 作为应用程序跨平台的首选，现已被应用于互联网、个人电脑、数据中心、超级计算机等各个领域。尤其是在服务器领域，大多数服务器应用程序都需要依赖于 Java 平台的支持[1]。Java 平台的核心是 Java 虚拟机。而目前，主流的 Java 虚拟机仅支持 x86 和 sparc 架构，尚未提供对于 MIPS 架构的支持，Java 虚拟机在 MIPS 平台上存在较大的功能和性能问题。为此，设计和实现一个高性能、高可靠的支持 MIPS 指令集的 Java 虚拟机具有十分重要的意义。

本文基于龙芯平台，给出了 MIPS 平台上 Java 虚拟机的实现方案。促进了 MIPS 高性能处理器在服务器领域的市场推广。

1.1 MIPS 64 位龙芯平台简介

自 2002 年 8 月，首片龙芯 1 号 XIA50 流片成功以来，龙芯的 CPU 产品大踏步迈进，针对不同的应用领域龙芯设计了不同的处理器。主要分三大类：针对嵌入式领域的龙芯 1 号系列产品、针对桌面应用领域的龙芯 2 号系列产品和专用于服务器领域的龙芯 3 号系列产品。本课题基于的硬件平台就是针对服务器应用领域的龙芯 3 号 MIPS 64 平台。

龙芯 3 号多核处理器现有两款产品，第一款产品是龙芯 3A 处理器，它是 RISC 架构，兼容 MIPS 64 位指令集，采用 65 纳米工艺，片内集成 4 个 64 位的 GS464[2]高性能处理器核，工作主频最高可达 1GHz，包含 4.52 亿个晶体管，最高峰值运算能力为 16GFLOPS，是我国第一个具有完全自主知识产权的四核处理器，已被用于早前的国产逸珑 8133 便携笔记本中。第二款产品是龙芯 3B 处理器，它是首款国产商用 8 核处理器，其晶体管数目为 5.826 亿个，主频最高达 1.05GHz，浮点峰值计算能力达到 128GFLOPS。我国首台万亿次高性能计算机“KD-90”就是采用龙芯 3B 八核处理器研制而成的。下面介绍龙芯 3 号系列处理器的特点：

龙芯 3 号多核系列处理器采用可伸缩的多核互连架构设计，在每个芯片上都集成了多个 cpu 核和大量的二级缓存。为了组成更大规模的系统，还通过高速 I/O 接口实现了多个芯片之间的互连，最多连接的处理器核可达 64 个。它的物理地址分布采用全局可访问的层次化寻址设计，以保证系统开发的扩展兼容。整个系统的物理地址宽度为 48 位。按照地址的高 4 位，整个地址空间被均匀的分布到 16 个节点上，即每个节点分配 44 位地址空间。最主要特点如下：

- 兼容 MIPS 64 位指令集，并基于龙芯平台进行了指令集扩展；

龙芯 3 号处理器系列完全兼容 MIPS 64 R2 体系结构，提供了其定义的全套指令。另外，为了提高常见应用的性能，还提供了对整点、浮点以及访存扩展指令；为了对越来越多的多媒体应用提供支持，龙芯提供了多媒体（SIMD）指令；为了提供在硬件层面对 x86 虚拟机的支持，还提供了 x86 虚拟机扩展指令。

- 采用动态调度技术、寄存器重命名技术、转移预测技术乱序执行技术；

寄存器重命名技术解决读后写和写后写相关，并用于例外和错误转移预测引起的精确现场恢复，GS464 分别通过 64 项的物理寄存器堆进行定点和浮点寄存器的重命名。动态调度技术根据指令操作数准备好的次序而非指令在程序中出现的次序来执行指令。转移预测技术是通过预测转移指令是否成功跳转来减少由控制相关引起的阻塞。

- 采用四发射超标量结构，两个定点、两个浮点和一个访存部件；

GS464 的每个浮点部件都可以全流水地执行 64 位双精度浮点乘加操作，可通过设置浮点指令的 fmt 域扩展执行 32 位和 64 位定点指令以及 8 位和 16 位多媒体加速 SIMD 指令。每个浮点单元有 64 个物理浮点通用寄存器。龙芯 GS464 浮点寄存器与 MIPS R2 浮点寄存器总体兼容，略有差别。龙芯 GS464 浮点通用寄存器沿用了 R10000 浮点寄存器的用法，体现在：当 CP0 的 Status 寄存器 FR 位为 1，表示系统中有 32 个 32 位的逻辑浮点通用寄存器；当 FR 为 0，表示只有 16 个 64 位的逻辑浮点寄存器。在物理实现上，32 个逻辑浮点通用寄存器通过重命名模块映射到物理浮点寄存器。

- 内置 128 位存储访问的访存部件，虚地址和物理地址宽度为 48 位；

龙芯 GS464 处理器核内置了内存管理单元（MMU）实现从虚拟地址到物理地址的转换。GS464 处理器核可以工作在三种模式下，分别是内核模式、管理模式和用户模式。不同的工作模式对应不同的虚拟地址空间：内核地址空间、管理地址空间和用户地址空间。即便所有虚地址空间为 64 位，但在 48 位物理地址模式下（PABITS=48），其最大支持的虚地址空间为 48（SEGBITS=48）位，即其支持的最大段为 256T（2^{48}）字节。

1.2 服务器应用程序

服务器应用程序主要分为文件服务器、数据库服务器和应用程序服务器。绝大多数应用程序服务器都是 Java 应用服务器。Java 应用服务器的种类繁多，按照服务器的功能，将其分为 JSP（Java Server Pages)服务器和 Java EE 服务器。

JSP 是一种动态网页技术标准。基于 JSP 开发的 Web 应用是跨平台的。主要的 JSP 服务器有：Tomcat 服务器、Bejy Tiger 服务器、Jonas 服务器、Jetty 服务器、Jrun 服务器、Resin 服务器和 Orion 服务器。

其中 Tomcat 服务器作为小型轻量级服务器的代表，由于其运行过程中占用的系统资源较小、且具有很好的可扩展性，对负载均衡与邮件服务等常用功能有很好的支持。再加上其免费并开放源码，已经成为中小型系统 JSP 开发的首选服务器。

JRun 服务器作为大型服务器和商用领域的代表。最先是由 Allaire 公司开发的，现在归 Adobe 公司所有。它是一款付费软件。它由 Java 服务器页面、企业版 JavaBean、Java servlet、Java 事务服务和 Java 消息服务五大模块组成。它的高级版专为机群服务器环境配置 JSP 和 servlet 应用程序。它还专为电子商务 Java 应用公司开发了企业版。

与 JSP 服务器相比，Java EE 服务器的功能更强。它作为一种中间件服务实现了企业级面向服务的体系结构和对 Web 应用程序的支持。

目前，国内具有代表性的 Java EE 服务器有东方通 TongWeb 服务器、金蝶 Apusic Application Server 服务器、宝兰德软件公司的 BES Application Server 服务器。

国外主流的的 Java EE 服务器有 Apache 软件基金的 Geronimo 服务器、Sun 开发的 Java System Application Server 服务器、IBM 公司的 Websphere 服务器、Oracle 公司的 Oracle9i/AS 服务器、Bea 公司的 Weblogic 服务器以及开源的 JBoss 服务器和 GlassFish 服务器。

这些 Java 应用服务器已经广泛应用于各行各业的数据存储、邮件服务、分布式应用以及网络服务中。而所有这些 Java 应用服务器都需要依赖于 Java 平台的支持。可见，龙芯服务器的推广，势必需要有一个稳定的、与业界兼容的 Java 虚拟机环境。

1.3 Java 平台介绍

Java 自 1995 年 5 月由 Sun Microsystems 公司推出以来，经过不断的技术更新和完善，已经成为跨平台应用软件开发的首选。现已广泛应用于互联网、移

动电话、游戏控制台、个人电脑、数据中心、超级计算机等各个领域。它具有卓越的通用性、平台可移植性、网络安全性。它拥有全球最大的开发者社区和最多的开发人员。近年来，随着全球云计算和移动互联网产业的迅速崛起，Java 由其自身显著的优势被赋予了更加广阔的发展前景。接下来介绍 Java 程序设计语言的特点和 Java 虚拟机的核心技术。

1.3.1 Java 程序设计语言特点

Java 语言[3]是一种基于 Unicode 字符集的面向对象的程序设计语言。它支持多线程和网络编程、自动内存管理、动态类型装载以及同步和异常处理。

Java 语言是一种强类型语言。Java 语言只有两种类型：基本类型和引用类型。其中，基本类型包括 boolean 类型、byte 类型、char 类型、short 类型、int 类型、float 类型和 double 类型。与 C 语言等其他语言不同的是 Java 规定了这些类型的大小。这保证了在所有平台下的数据一致性。

Java 语言是一种动态类型语言。Java 类在程序运行期间动态加载和链接。这些类的来源可以是网络、文件、甚至是运行过程中在内存中生成的类。Java 类的动态加载是基于 Java 虚拟机的动态类加载机制实现的。这种机制提供了对 Java 类的延迟载入、Java 类型安全检查和自定义类装载器的支持。这种完全动态的特征让 Java 程序变得非常灵活的同时，给 Java 虚拟机的实现增加了难度。

Java 语言支持多线程编程。这些线程独立执行代码，Java 多线程是通过以下三种方式得到支持的：多硬件处理器同时处理不同的线程、将单个处理器进行时间分配以及将许多硬件处理器进行时间分配实现多线程的支持。因此，在虚拟机设计时需要根据不同的硬件环境实现不同的多线程处理方式。

Java 语言提供了多种线程间通信机制。其中最基本的是同步和消息传递，Java 线程同步使用监视器（monitor）来实现。每个 Java 对象关联一个监视器。Java 线程通过加锁和解锁监视器来实现互斥。在虚拟机的实现中，需要考虑同步操作带来的时空开销，并做相应的优化。消息传递是通过调用线程的 wait 方法，notify 方法以及 notifyAll 方法来实现的。

1.3.2 Java 虚拟机核心技术

Java 虚拟机[4]是基于 Java 语言的特点而实现的。为了很好的支持 Java 语言，虚拟机需要实现几个主要机制：动态类装载机制、内存管理机制、锁机制、编译机制以及异常处理机制。随着软件开源的盛行，很多虚拟机开放了源代码，这使得对于虚拟机的研究和开发成为可能。为此，基于 Java 虚拟机的各项研究全面展开。下面，主要介绍 Java 虚拟机的核心相关技术。

- 动态类装载机制

动态类装载机制是实现 Java 运行时加载类以及网络编程的基础。它的主要原理就是在程序真正需要时查找并装载类型信息，并进行类型正确性检查，转换类型符号引用为直接引用，最后进行变量初始化。动态类装载机制为用户自定义类装载器提供调用接口。用户可以通过实现此接口，进行自定义类装载器实现。此外，动态类装载器支持多命名空间，可以为不同的软件组件提供单独的命名空间。有关动态类装载器的研究主要集中于对类型安全检查的研究。这是由于一方面在程序实际执行前，类的类装载器是无法确定的，因为类装载机制会根据用户编写的代码的逻辑互相委托进行加载动作。而另一方面在类还未使用前进行提前加载是不受程序欢迎的。一种通用的解决方案是采用一种约束机制，在类加载时进行约束检查，如果有必要的话发布有歧义的命名空间[5]。

- 内存管理技术

Java 虚拟机的内存管理[7]分为存储模型和存储管理两部分。存储模型用来存储 Java 虚拟机运行时的对象，方法，常量等信息。Java 虚拟机将存储空间分为堆空间和栈空间。存储管理针对堆空间进行对象的分配和回收。也就是通常说的垃圾回收机制。目前，垃圾回收技术已经非常成熟。有关垃圾回收算法的研究主要分为 Stop-The-World 垃圾回收算法和渐进式垃圾回收算法和并发垃圾回收算法[8]。所谓 Stop-The-World 是指在垃圾回收期间虚拟机必须停止执行直到垃圾回收结束才能够恢复运行。基于 Stop-The-World 的经典算法有：引用结束算法、标记-清楚算法以及半复制算法等[9]。分代垃圾回收技术是目前 Java 虚拟机常用的垃圾回收技术，它根据对象的生存周期将存储区分为不同的区域，并根据不同代的特点采用不同的垃圾回收算法实现。渐进式垃圾回收技术允许中断垃圾收集过程，当垃圾收集线程再一次被唤醒时，垃圾收集从被中断的地方重新开始执行。它需要保证一次垃圾收集的时间不超过某个阈值。并发式垃圾收集与渐进式垃圾收集原理一致，只是并发垃圾收集是基于多核处理器对内存数据访问一致性，比较复杂。

- 锁机制

锁机制[10]是 Java 语言多线程编程实现的关键。它主要用于多线程之间数据共享。Java 虚拟机为每个对象分配一个监视器。在 Java 虚拟机中通过加锁和解锁实现对监视器的控制。目前，大多数研究都是致力于降低同步原语的执行开销。Java 虚拟机中的锁分为三个级别，分别是重锁、轻锁和偏向锁[11]。重锁是通过操作系统调用来实现的；轻锁是以假设大多数程序中锁的获取是没有竞争的为前提实现的，为了尽量避免对操作系统的调用在进入和退出监视器时使用原子操作（CAS 指令）达到同步的目的；偏向锁则是在轻锁的基础上进一步

假设大多数监视器都是被一个线程加锁和解锁的，它允许线程在进入和退出时不使用原子操作。但是当另一个线程企图进入已经加了偏向锁的监视器时，需要执行偏向撤销。该优化是建立在消除原子操作所带来的效益远高于执行偏向撤销所带来的性能损失的基础上而实现的。

- 异常处理机制

异常处理机制并非 Java 程序特有，异常处理的思想早在 1975 年就由 Goodenough 提出在程序语言中支持异常处理。并在多种语言中都实现了异常处理机制，如 Python，PHP，C++，ML，Ada 等。异常处理的核心是从一个异常被抛出到最后异常被捕获的过程中进行一系列查找和跳转操作，最后转到相应的异常处理代码处，执行异常处理。整个过程需要编译器和运行时异常处理支持函数的紧密配合以及对栈进行展开和或者栈切换的操作实现[12]。栈展开是指，当异常抛出时，将栈帧展开搜索相应的异常处理器。当异常处理器就在异常抛出的当前函数中时（正常路径），没有太多的开销，然而当异常处理器处于调用路径上时（非正常路径），需要多次展开栈查找匹配的异常处理器，开销相当高。栈切换是指在已注册的异常处理列表中搜索相应的异常处理器。对于正常路径需要一定的注册和注销开销，但是通过注册列表可以减少在非正常路径上找到正确异常处理程序的开销。栈展开在正常路径比栈切换更有效。

1.3.3 Just-in-time 编译

编译的本质是将程序语言翻译成可执行的格式。传统的翻译方式有两种：静态编译和解释。静态编译是指在翻译的过程中先将源指令转换成一种中间格式，然后进行进一步的翻译和优化。最终翻译后的代码更适合后续的执行。解释跟静态编译一样进行翻译，但它消除了中间步骤，代码经过翻译后马上进行执行。

静态编译后的程序运行速度很快，尤其是在它们被编译成一种可以直接在硬件上执行的形式时。但是静态编译需要在程序的执行过程中投入很多的时间进行程序分析和优化。解释程序由于它选择了一种比机器码高级的表示。可以带很多隐含的信息。解释程序如果采用一种机器无关的表示，如高级源码或者虚拟机，它可以在不同的目标机器上运行。

Just-in-time 编译（简称 JIT 编译）综合了静态编译和解释的上述优点。JIT 编译器概念最早源于 1960 年，McCarthy[13]在他发表的一篇关于 LISP 的文章中提出一种足够快的不需要保存输出结果的一种编译机器码的思想。从那以后，JIT 编译器就一直被应用于软件系统中。Java 虚拟机中 JIT 编译器的使用，促进了 JIT 编译技术的发展。JIT 编译指程序开始启动后的任何动态编译执行行为。JIT 编译的实现需要解决如下三方面的共性问题[14]：

1. 二进制代码生成。代码刚生成时有些信息还无法获取到，如向前分支跳转目标。一旦发现分支目标已生成，需要将该目标回填到原来的位置。

2. 高速缓存一致性。CPU 运算速度往往超过了存储速度。为了弥补这一差距，现代 CPU 设计都会内置一个小的、快的存储器，该存储器的内容可能与主存内容有时会不同步。当代码动态生成时，要确保在执行前所有数据都同步到主存中。对于多个 CPU 共享一个主存的情况会更加复杂。

3. 硬件或者操作系统可能限制可执行代码的驻留区。如生成的代码放在数据区，但是某些平台默认数据区不允许执行。

JIT 编译的实现依赖于具体的编译语言。不同语言实现方式不同。根据对某些属性的实现方式分为如下几类：

1.根据调用方式分为显式调用编译器和隐式调用编译器。当用户需要在运行时执行某些动作时显式的触发编译成为显示调用编译器。而与之相反的是，隐式调用编译器是指程序根据当前执行行为调用编译器，对用户透明。

2. 根据执行语言的种类分为单语言 JIT 系统和多语言 JIT 系统。JIT 系统通常涉及两种语言：源语言和目标语言。单语言 JIT 系统只能执行一种语言（即源语言和目的语言相同，如它只进行优化时）。如果它可以执行两种或者两种以上语言叫多语言 JIT 系统。

3. 根据编译器的执行方式将其分为并发编译器和非并发编译器。如果编译器的执行是由程序控制的，程序根据执行意图启动和禁止编译，此时编译器相当于一个子程序，那么它是非并发编译器。反之，一个并发编译器可以跟程序同时执行。使用另一个线程。或者甚至是另一个处理器。

Java 虚拟机中的 JIT 编译为了减少编译开销，只有确定方法会执行才编译一个方法。因为并非所有的方法都会执行，通常一个类中被执行的方法不到一半。JIT 编译尽可能执行少的代码，保证其编译速度足够快，以补偿花在编译上的时间。此外，减少编译启动延迟也同样重要[15]。

Java 虚拟机中的 JIT 编译器用于将 Java 字节码编译为本地代码。

JIT 编译需要找到方法生成高效的代码，而不像传统静态编译器一样使用昂贵的优化。此外，编译代码需要实现 JVM 规范。为此，Java 虚拟机中的 JIT 编译器实现需要考虑如下几方面的问题：

1. 降低编译开销。传统的编译需要构造一个中间表示。由于 Java 字节码本身就是一种中间表示，所以 JIT 编译可以消除中间表示所带来的开销。为了方便解释，Java 字节码被设计为紧凑的表示。虽然他们不是理想的中间表示，但是它们可以很容易的用于中间表示中。编译器中间表示应该保留所有的源代码级别的信息相关的优化和本地代码生成，同时隐藏无关紧要的语法细节。Java

字节码包含了类源文件的所有信息。在编译后只有少部分信息会丢失，如 boolean 类型转变为 int 型。

2. 生成高效代码。Java 操作数栈的使用限制了字节码的特定顺序。而在基于寄存器的处理器中，可以对字节码进行排序，生成更高效的代码。如将常量压入操作数栈，接着将其弹出，进行计算。如果在基于寄存器的处理器中，可以直接将常量保存到寄存器中，避免压栈和出栈开销。此外，可以为特定字节码选择合适的机器指令以及去除字节码中包括的冗余操作，如数组越界检查、空检查等。

3. 保证正确性。Java 编译执行需要遵循 JVM 规范，除了这点外，其他跟别的编译没有什么区别。JVM 规范定义了很多细节，如一个线程执行过程中遇到栈空间溢出，JVM 必须抛出一个特殊的异常报告这个问题；用户在代码中指定当类被首次调用时必须执行指定的初始化代码等。这些显著的增加了 JIT 编译器的复杂性。

1.4 MIPS 64 位 Java 虚拟机移植的必要性

目前，在龙芯平台上可运行的 Java 虚拟机仅支持 MIPS 32 位版本[16][17]。受到 MIPS 32 位体系结构以及 Linux 内核的固有限制，任何应用程序能够使用的存储空间不超过 1GB。而目前大多服务器应用程序要求非常高的吞吐量以及并发量，1GB 的存储空间显然已经成为限制其性能提升的一个瓶颈。另外，随着处理器技术的迅猛发展，64 位多核处理器已经应用于服务器和高性能计算机应用中。原来基于 32 位指令集的应用程序已经无法充分发挥 64 位现有硬件资源环境的优势。为此，设计一个稳定、高效的可支持 MIPS 64 位指令集的 Java 虚拟机非常必要。主要体现在如下四个方面：

1. 可以使用超过 4GB 字节以上的存储空间，克服 32 位体系结构的固有缺陷。理论上，64 位可寻址能力达到 16EB（1EB = 2^{60}）字节。但实际中，没有如此大的内存。随着近年来内存技术的不断发展和更新，内存的容量不断扩大，价格也越来越便宜。可用物理内存空间呈不断上升趋势。另外，随着应用的不断丰富以及用户访问量的指数增长，应用程序对存储空间的需求越来越高。加上 64 位操作系统对于 64 位应用程序的很好支持等因素。未来，64 位软硬件环境将是一个必然趋势。

2. 支持 64 位宽的运算能力，比 32 位架构具有更高的计算效率。64 位系统下通用寄存器的宽度变为原来的 2 倍。原来对于一个大于 32 位数据的取数和运算均需要 4-8 条指令完成，现在可在一到两条指令下完成，大大加快了运算速度。另外，64 位宽度可以保留更多的浮点运算精度，提高了浮点运算的精度。

3. 由于存在上述优势，64 位平台已经成为当前主流的计算架构。为了充分发挥 64 位处理器的优点，Java 虚拟机也有必要进行相应的扩展和改进。

综上所述，为了更好的提高 Java 虚拟机在 MIPS 64 位平台上的性能，设计和实现一个稳定高效的基于 MIPS 64 指令集的 Java 虚拟机具有十分重要的意义。

1.5 现有主流 Java 虚拟机

目前，基于桌面和服务器领域最主流的有三大商用虚拟机，分别为 Oracle 于 2010 年收购 Sun 公司的 HotSpot 虚拟机、于 2008 年收购 BEA 公司的 JRockit 虚拟机和 IBM 公司的 J9 虚拟机。此外，在高性能虚拟机领域最具代表性的是 Azul Systems 公司的 Azul 虚拟机和 BEA 公司的 Liquid 虚拟机。近三年来，随着移动数码设备的迅猛发展，Apache 软件基金下的 Apache Harmony 虚拟机和 Google 公司的 Android Dalvik 虚拟机这两款虚拟机对 Java 世界产生了非常大的影响和挑战。

- **高性能 Java 虚拟机**

Azul 虚拟机[18]是 Azul Systems 公司开发的专门运行于 Vega 硬件系统上的 Java 虚拟机，它可以管理至少 10 个 CPU 和数百 GB 字节的内存，实现了可控时间的垃圾收集器和为专有硬件优化的线程调度。

Liquid 虚拟机[19]是 BEA 公司开发的可直接运行在 Hypervisor 系统上的 Java 虚拟机，它实现了一个专用操作系统的必要功能可以越过通用操作系统直接控制硬件，最大限度的发挥硬件的能力，提升 Java 程序的性能。但由于其专用于特定的硬件平台，通用性不是很好，不适于龙芯虚拟机的选择。

- **新型 Java 虚拟机**

Harmony 虚拟机[20]是由 Apache 软件基金开发的与 JDK1.6 兼容的 Java 虚拟机，它包含自己的虚拟机和 Java 库，可以运行常见的 Java 应用，但由于其未经 TCK（Technology Compatibility Kit）授权，再加上 OpenJDK 的开源极大的削弱了 Harmony 的推广和使用。但其 Java 库的代码被 IBM 和 Google Android SDK 广泛采纳。

Dalvik 虚拟机[21]是 Android 平台的核心组成模块。它没有遵循 Java 虚拟机规范，不能直接执行 Java class 文件。所以它不是一个 Java 虚拟机。但它可以使用大部分 Java API 编写 Java 应用程序，并将 class 文件转化为 dex（Dalvik Executable）文件进行执行。这两款虚拟机由于其未经授权和未遵循 Java 虚拟机规范，不适合龙芯 Java 虚拟机的选择。

- **主流 Java 虚拟机**

JRockit 虚拟机[22]最早由 Appeal Virtual Machines 公司开发后于 2002 年被 BEA 公司收购的一款专用于服务器领域的高度优化的虚拟机。由于其专注于服务器应用，对于启动速度要求不高，因此它只有编译器。此外，其垃圾回收器和 MissionControl 服务套件在众多 Java 虚拟机中处于领先水平。

J9 虚拟机[23]扩展自 IBM Ottawa 实验室的 SmallTalk 虚拟机，是一款从服务器到桌面再到嵌入式领域的多用途虚拟机，主要与 IBM 产品搭配在 IBM AIX 和 z/OS 平台上使用。这两款虚拟机由于其不支持 MIPS 架构和代码不开源性都不适合在龙芯上实现。

HotSpot[24]虚拟机是 Sun JDK 和 OpenJDK 中使用的虚拟机，是目前使用最广的虚拟机。它最初由一家名叫 Longview Technologies 的公司为 Strongtalk 语言开发的以支持面向对象程序设计语言为目的而设计后被 Sun 公司于 1997 年收购，并综合了 Sun 最初开发的 Classic VM 和 Exact VM 两款商用虚拟机的优点而设计的。HotSpot VM 的特点是它的热点代码探测技术，它通过对方法的调用次数和方法中的循环计数，找出最优编译价值的代码，通知编译器编译该方法。HotSpot VM 通过协同解释器和编译器恰当工作有效的取得了最优程序响应时间和最佳性能的平衡。此外，由于即时编译的时间压力较小，可以引入更多的代码优化技术，产出更高质量的代码。

自 Sun 公司于 2006 年在 JavaOne 大会上宣布开源 Java 以来，JDK 包括 HotSpot 在内的各个部分在 GPL 协议下公开了源码，并建立了 OpenJDK 项目有 OpenJDK 社区维护。自此，HotSpot 便成为了 Sun JDK 和 OpenJDK 两个实现非常接近的项目的共同虚拟机。

OpenJDK 6[25]是 OpenJDK 开源项目之一，其内置的 HotSpot 虚拟机结合了国内外现有最优的虚拟机技术，已经成为目前最广泛使用的 Java 虚拟机。它与 SunJDK6 完全兼容，所有采用 SunJDK6 Java 规范开发的应用软件可以在 OpenJDK 6 上面直接运行。是龙芯 Java 虚拟机实现的首选平台。

1.6 本文研究内容与研究工作

本文在已有 32 位 Java 虚拟机的基础上，经过一年的实践与研究，完成了 64 位 OpenJDK 6 在龙芯 MIPS 上的实现，并基于龙芯平台进行了性能优化。具体工作如下：

1. 分析移植 64 位 Java 虚拟机到龙芯平台的必要，研究虚拟机相关技术，为后期工作做准备。

通过阅读相关文献，了解 Java 语言的基本特点，研究了 Java 虚拟机的各个模块，包括类装载机制、内存管理技术、锁机制、编译器实现原理、异常处理

机制等。并分析了目前流行的几种 Java 虚拟机实现，并对他们分别作了比较和分析。最终选择 OpenJDK 6 作为龙芯平台 Java 虚拟机。这主要是基于其成熟的虚拟机实现技术和与当前主流 JDK6 规范兼容，更重要的是由于其完全开放源码，这为虚拟机的移植和研究奠定了基础。

2. 研究 MIPS 64 平台的特性，并移植 Java 虚拟机解释器部分。

对 MIPS 64 平台的指令集、寄存器约定以及整个体系结构做了系统的研究和分析，并与 X86 体系结构做了对比。实现了 Java 虚拟机 MIPS 64 位中的汇编器部分，并对模板解释器、方法调用入口、寄存器、Java 栈帧以及本地方法调用接口等实现做了移植。使得 OpenJDK 6 可以在龙芯平台下进行纯解释执行。

3. 分析 JIT 编译器的实现机制，实现了 MIPS 64 位虚拟机的即时编译器。

通过对 Java 虚拟机的编译器构成模块的分析，以及对平台相关的 LIR 中间表示模块的重点研究，实现了编译器中的方法调用接口、编译器寄存器分配、C1 编译器实现、与解释器进行转换的相关接口、异常处理、去优化实现接口、Safepoint 安全点等。最终实现了 OpenJDK 6 MIPS 64 位编译器模块的移植。

4. 采用标准测试工具 SPECjvm2008 测试 MIPS 64 位 Java 虚拟机的性能。

本文移植的 MIPS 64 位 Java 虚拟机已经可以稳定的运行于龙芯平台。分析 32 位 Java 虚拟机在某些 SPECjvm2008 测试集上的性能优势，分析 Java 虚拟机运行时行为，找出可以改进的方面进行性能优化与分析。增加压力测试和稳定性测试，提升 64 位 Java 虚拟机的稳定性。

1.7 本文的组织结构

接下里的章节，将介绍在上述研究过程中所采用的相关技术和所做的研究工作，最后进行总结和展望，并提出建设性意见。

第一章，绪论。介绍了本课题的研究背景以及国内外的相关研究技术，并对作者的研究内容和工作进行了说明。

第二章，OpenJDK 6 在 MIPS 64 上的移植实现。分别介绍了 OpenJDK 6 寄存器分配移植、基于模板的解释器移植、JIT 编译器的移植以及运行时环境的移植等内容。

第三章，基于龙芯 3A 平台 OpenJDK 6 的优化。本章分析了 OpenJDK 6 的性能瓶颈，并实现了 OpenJDK 6 中的 3 项关键优化。分别是获取 Java 线程控制结构首地址优化、JNI 接口调用优化和 safepoint 页面分配优化。

第四章，OpenJDK 6 在 MIPS 64 平台上的性能评测。本章介绍测试 Java 虚拟机的软硬件平台和测试工具，接着分析了第三章中的三项优化效果，最后对本文实现的 64 位 OpenJDK 6 虚拟机进行整体测试，并与 32 位 Java 虚拟机进行

了性能比较与分析。

第五章，总结与未来工作。本章对整篇文章做了总结，同时提出了下一步的工作方向和建议。

中国科学院计算技术研究所 鄢銮 2025-03-14 12:09:09

第二章 MIPS 64 位 Java 虚拟机的实现

本章基于 OpenJDK 6 在龙芯平台上实现了 MIPS 64 位的 Java 虚拟机。开篇分析了 OpenJDK 6 的结构和功能模块，并基于该结构分析，对 OpenJDK 6 在 MIPS 64 位中需要实现的功能模块进行了阐述。 接着从四个方面介绍了 OpenJDK 6 实现。分别是：

1. MIPS 64 位虚拟机寄存器分配实现。首先分析了 MIPS 64 位寄存器的使用策略，接着基于此分析从寄存器接口、整型寄存器、浮点寄存器、栈顶缓存寄存器和特殊寄存器等方面进行了实现。

2. 模板解释器的 MIPS 64 位实现。首先分析了模板解释器平台相关的部分，接着分别从模板生成函数和解释生成器两个方面进行了实现。整个过程涉及到几乎所有 Java 字节码的模板生成实现和所有进入方法入口前代码段的移植。

3. JIT 编译器的 MIPS 64 位实现。JIT 编译器中与平台相关的实现包括：用于实现寄存器与中间表示操作数映射的编译栈帧映射、生成中间表示的 LIR 生成器、将中间表示转换为本地汇编指令的 LIR 汇编器。整个过程涉及到所有的 LIR 指令的汇编生成实现。

4. 运行时环境实现。运行时环境实现了对解释执行和编译执行转换的支持，对调用者保存寄存器的实现支持，以及对方法调用和值传递机制的支持。提供了 I2C 适配器和 C2I 适配器用于实现执行模式的切换。

最后，通过实际案例分析了调试过程中的难点，并给出了具体的解决思路，可作为同类问题的参考实现方案。

2.1 OpenJDK 6 结构

OpenJDK 6 由提供 Java 运行环境和基本类库接口的 jdk 模块、处理 XML 文档编程接口的 jaxp 模块、处理 Web 服务接口的 jaxws 模块、提供面向对象应用程序体系规范接口的 corba 模块和虚拟机实现模块 hotspot 五个模块组成。前四个模块提供 Java 虚拟机运行过程中需要的外围环境和 JDK 工具。hotspot 模块是 OpenJDK 6 的核心模块。

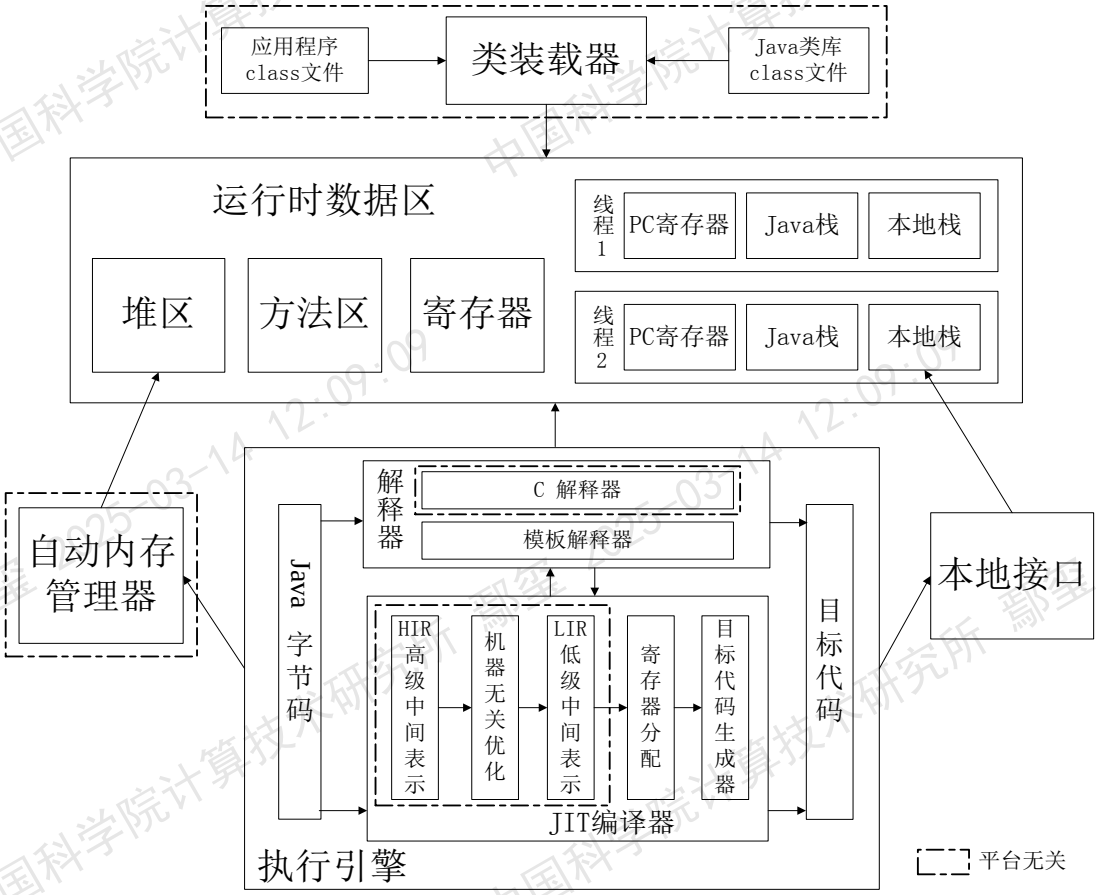


图 2- 1 HotSpot 结构图

hotspot 的整个结构如图 2-1 所示，它由类装载器、执行引擎、本地接口、运行时数据和自动内存管理器五个部件构成。

类装载器负责查找并装载Java程序运行过程中的应用程序class文件和Java API class 文件，检查 class 文件的合法性，验证导入类型的正确性，在运行时数据区上分配该类所需要的内存空间并进行初始化。

运行时数据区由所有线程共享的堆区、方法区和寄存器以及 Java 栈和本地方法栈构成。堆区保存 Java 虚拟机运行过程中的类实例和数组实例。方法区存放该类型及其超类型的全限定名、类型访问修饰符、类变量、字段信息、常量池信息等。寄存器用于整个虚拟机的汇编实现代码中。其中 PC 寄存器在 Java 方法的执行中指向下一条指令地址，在本地方法的执行中未定义。Java 栈保存了 Java 线程的运行时状态，以栈帧为单位进行压栈和出栈。本地方法栈保存本地方法运行过程中的信息，由本地方法接口访问。

本地接口是对 Java 语言中 JNI 接口的实现，其执行过程需要在本地方法栈中创建对应的本地方法栈帧、进行参数传递、返回值处理、同步、safepoint 检测等。

自动内存管理器管理运行时数据区的堆区，它负责对象内存空间的分配，并对无用的对象进行垃圾清空和内存回收。自动内存管理机制降低了软件设计的复杂性，使得软件设计更加模块化，有利于软件的开发和维护，提高了软件开发的效率。

执行引擎是 hotspot 的核心部件。执行引擎负责将 Java 字节码转换为本地汇编代码进行执行。字节码可以通过解释器解释执行也可以被 JIT 编译器编译执行。解释器对字节码进行逐条解释执行。编译器将解释器收集的热点方法整体编译成本地汇编后执行整个方法。在 Java 虚拟机的整个运行期间，解释器和编译器交替执行字节码，直到程序终止。

解释器的原理是取一条源指令，将其解释为相应的目标指令，执行指令，修改运行时环境的状态。再取下一条指令，如此循环直至程序运行结束。解释器的实现因不同的系统需求，不同的软硬件环境，不同的设计目标而不同。如图 2-1 所示，OpenJDK 6 中实现了两种解释器。一种是 C 解释器，该解释器完全使用高级语言 C 语言进行解释，可以不加移植的运行于支持 C 编译器的任何平台。也正是由于被间接的转换为本地二进制执行，其运行速度比较慢。另一种是模板解释器。该解释器将目标字节码直接解释为本地汇编指令。由于其避免了中间转换过程，执行速度很快。

编译器的执行过程是首先将 Java 字节码转换为高级中间表示形式，高级中间表示是一种使用静态单一赋值 SSA[26]（Static Single Assignment）格式的，基于控制流图 CFG[27]（Control Flow Graph）的平台无关的表示方式。接着进行与机器无关的优化，包括空检查消除、条件表达式消除、全局变量编号[28]等。再接着将高级中间表示转换为与汇编指令非常接近的低级中间表示，进行寄存器分配[29]，最后目标代码生成器将低级中间表示转换为平台相关的目标代码。

2.2 实现模块相关工作分析

本节基于前文所述的 OpenJDK 6 的结构，对其各个模块的平台相关性进行分析，并阐述平台相关模块所需要进行的移植实现。

OpenJDK 6 中与平台相关的模块为 hotspot 模块。由图 2-1 hotspot 结构可知，除了类装载器、自动内存管理器、C 解释器和 JIT 编译器中的前三个阶段外，其他实现都与目标平台紧密相关。

尽管 MIPS 64 体系结构的设计保留了所有的 MIPS 32 位指令，可以实现 100% 兼容。但是，由于其寄存器宽度和地址空间的改变，导致所有的算术运算指令、移位指令和存取指令都进行了重新设计。相应的整个 hotspot 中与平台相关的部分都需要针对 MIPS 64 位结构进行改造。

具体从以下四个方面展开：

1. 运行时数据区中寄存器的实现。由于 MIPS 64 平台对于寄存器的修改非常大，且 Java 虚拟机执行过程中高度依赖目标寄存器，所以需要对寄存器进行整体移植。具体对整型寄存器和浮点寄存器的移植包括：寄存器接口移植、针对整型寄存器约定变化所做的移植、针对浮点寄存器约定变化所做的移植、对寄存器压栈与出栈所做的移植、特殊用途寄存器的移植。

2. 执行引擎中模板解释器的实现。由于 MIPS 64 位中寄存器宽度、地址位宽的增加和算术指令、移位指令、访存指令的重新定义，这些改变导致 MIPS 64 位解释器中使用汇编实现的模板生成函数和模板解释生成器都需要移植。模板生成函数的移植包括针对地址空间宽度变为 64 位所做的移植和针对 Java 数据类型 long 型、double 型所做的移植。模板解释生成器的移植包括方法入口地址的移植、Java 栈帧的移植和参数传递的移植。

3. 执行引擎中 JIT 编译器的实现。由于 JIT 编译器低级中间表示的生成依赖于目标寄存器，所以，MIPS 64 位中需要对于生成低级中间表示的 LIR 生成器进行实现。此外，由于 MIPS 64 位寄存器和指令的变动，导致实现寄存器映射的编译栈帧映射机制的重新改造，以及需要目标平台指令支持的低级中间表示向本地汇编指令的转换实现需要重新实现，这涉及到所有的 LIR 指令的汇编生成，包括算术操作、方法调用操作、数组拷贝操作等。

4. 支持解释、编译执行转换的运行时环境的实现。MIPS 64 位中新增了参数寄存器，为此，重新设计了运行时方法调用的传参机制，增加参数寄存器的个数，提高了传参效率。针对 64 位数据宽度的提升和 MIPS 64 位的新增指令，原来对于 long、double 型数据传递的机制需要重新实现、解释器和编译器适配器也需要进行改造。

上述四方面的实现工作的具体原理和实现方法，在本章后续的各节中进行具体阐述。

2.3 MIPS 64 位虚拟机寄存器分配实现

Java 虚拟机是一种基于栈的指令集体系结构。它是以能够提供到各种主机平台的可移植性为主要目的而设计的一种特殊的系统接口。因此，它对主机平台寄存器没有特殊的要求，它可以运行于寄存器很少的主机平台，如嵌入式平台；也可以运行于寄存器数量非常丰富的主机平台，如本文的移植平台（龙芯 MIPS 64 平台）。本小节首先介绍 MIPS 32 位平台与 MIPS 64 位平台寄存器使用约定的区别，接着介绍基于这些差别所做的移植工作。

2.3.1 MIPS 64 位寄存器规范

32 位 Java 虚拟机实现中采用 MIPS o32 寄存器约定规范[30]，由于 64 位 CPU 体系结构设计更改，为与当前体系结构保持一致，64 位 Java 虚拟机实现中遵循 MIPS n64 寄存器使用规范。与 o32 约定相比，MIPS n64 规范在寄存器位宽和使用约定方面有很大的不同。下面分别对整型寄存器和浮点寄存器做出说明：

MIPS 64 位整型寄存器个数与 MIPS 32 相同，都是 32 个，但其位宽变为 64 位，是 MIPS 32 位整型寄存器的两倍。其寄存器使用约定变化不大，只将原 MIPS 32 位中的 4 个临时寄存器作为现在 MIPS 64 中的 4 个参数寄存器，用于传递第 5～8 个参数。表 2-1 给出了 MIPS 32 与 MIPS 64 整型寄存器约定区别。

表 2- 1 MIPS32 与 MIPS64 整型寄存器约定对比表

<table>
<tr><th rowspan="2">寄存器编号</th><th colspan="2">MIPS o32</th><th colspan="2">MIPS n64</th></tr>
<tr><th>名称</th><th>约定</th><th>名称</th><th>约定</th></tr>
<tr><td>$0</td><td>zero</td><td>始终为 0</td><td>zero</td><td>始终为 0</td></tr>
<tr><td>$1</td><td>at</td><td>汇编编译器使用</td><td>at</td><td>汇编编译器使用</td></tr>
<tr><td>$2，$3</td><td>v0，v1</td><td>函数返回值</td><td>v0，v1</td><td>函数返回值</td></tr>
<tr><td>$4-$7</td><td>a0-a3</td><td>函数参数寄存器</td><td>a0-a3</td><td>函数参数寄存器</td></tr>
<tr><td>$8-$11</td><td>t0-t3</td><td rowspan="3">临时寄存器</td><td>a4-a7</td><td>参数寄存器</td></tr>
<tr><td>$12-$15</td><td>t4-t7</td><td>t0-t3</td><td rowspan="2">临时寄存器</td></tr>
<tr><td>$24,$25</td><td>t8,t9</td><td>t8,t9</td></tr>
<tr><td>$16-$23</td><td>s0- s7</td><td>存储寄存器</td><td>s0- s7</td><td>存储寄存器</td></tr>
<tr><td>$26,$27</td><td>k0,k1</td><td>中断/陷入使用</td><td>k0,k1</td><td>中断/陷入使用</td></tr>
<tr><td>$28</td><td>gp</td><td>全局指针</td><td>gp</td><td>全局指针</td></tr>
<tr><td>$29</td><td>sp</td><td>堆栈指针</td><td>sp</td><td>堆栈指针</td></tr>
<tr><td>$30</td><td>s8/fp</td><td>帧指针或存储寄存器</td><td>s8/fp</td><td>帧指针或者存储寄存器</td></tr>
<tr><td>$31</td><td>ra</td><td>子程序返回地址</td><td>ra</td><td>子程序返回地址</td></tr>
</table>

MIPS 64 位浮点寄存器有 32 个 64 位的寄存器，而 MIPS 32 位中只有 16 个 64 位寄存器（或者从某种意义上说是，32 个 32 位寄存器），这导致 32 位下浮点寄存器只能成对使用。由于 MIPS 64 位浮点寄存器是专门针对 CPU 中的 64 位浮点单元而设计的，所以其寄存器使用约定变化很大。表 2-2 列出了 o32 与 n64 浮点寄存器使用约定区别，主要体现在：

1. 参数寄存器由原来的 2 个增加到 8 个。
2. 存储寄存器由原来的 6 个增加到 8 个。
3. 临时寄存器由原来的 6 个增加到 14 个。

表 2- 2 MIPS32 与 MIPS64 浮点寄存器约定对比表

MIPS 浮点寄存器使用约定	浮点寄存器编号	
	MIPS o32	MIPS n64
返回值寄存器	$f0, $f2	
参数寄存器	$f12, $f14	$f12-$f19
存储寄存器	$f20, $f22, $f24, $f26, $f28, $f30	$f24-$f31
临时寄存器	$f4, $f6, $f8, $f10 $f16, $f18	$f1, $f3 -$f11, $f20-$f23

2.3.2 寄存器分配策略实现

由于上节提到的寄存器使用规范上的变更，在 OpenJDk 6 的移植中需要对以下几个方面进行相应的移植：

1. 寄存器接口实现

虚拟机通过寄存器抽象接口调用目标平台的寄存器。为此，Java 虚拟机的移植必须实现该寄存器抽象接口。在 MIPS 64 位的移植中，定义了整型寄存器和浮点寄存器分别实现寄存器抽象接口。主要对寄存器名称、寄存器编号值、寄存器别名以及寄存器的个数进行了重新定义。

在 Java 虚拟机实现中，规定寄存器的位宽为 32 位，由于 MIPS 64 中每个寄存器的宽度为 64，所以在 64 位 Java 虚拟机中寄存器的个数为原 32 位 Java 虚拟机的两倍。

2. 针对整型寄存器约定变化所做的移植

Java 虚拟机中基于模板的解释器、编译器中 LIR 汇编代码生成实现、方法调用入口等都使用纯汇编语言实现，因此在实现过程中需要考虑寄存器的使用。如：原 32 位移植中存储临时变量可以采用 t0-t9 这 10 个临时寄存器，而由于

MIPS 64 约定的改变，现在只能使用 t0-t3,t8,t9 这 6 个寄存器。原来参数传递只能在寄存器中传递 4 个参数，其余参数需要在栈中传递，而现在可以在寄存器中传递多达 8 个参数。

此外，由于整型寄存器宽度变为 64 位，原来保存一个 64 位 long 型数值需要两个寄存器，现在只需要一个。

3. 针对浮点寄存器约定变化所做的移植

Java 虚拟机中对于 float、double 类型的操作所使用的寄存器为浮点寄存器。同整型寄存器一样，所有对于临时寄存器、存储寄存器、参数寄存器的使用都需要按照 n64 约定使用。

此外，由于浮点寄存器宽度变为 64 位，原 32 位虚拟机实现中对于一个 64 位 double 型的数值存储需要两个浮点寄存器，现只需一个。

4. 对寄存器压栈与出栈所做的移植

当 Java 虚拟机在编译执行过程中调用运行时方法时，需要将当前所有非调用者保存寄存器（包括整型和浮点寄存器）压入栈中，在运行时方法执行结束返回时，恢复这些寄存器的值。由于 MIPS 64 位中寄存器的宽度变为 32 位的 2 倍，所以其需要的栈空间也要变为原来的 2 倍。为此，在寄存器压栈和出栈时，需要修改栈偏移由原来的偏移 4 个字节变为 8 个字节。

5. 栈顶缓存寄存器移植

由于大多数情况下，压入 Java 栈中的数值，在接下来就会被使用。而 MIPS 架构所有的运算、移位等指令都是基于寄存器进行的。这样刚刚压入栈中的数值，紧接着使用时就需要出栈，这样会造成冗余的压栈、出栈操作。为了减少性能开销，会直接将值保存在栈顶缓存寄存器中。分别预留两个整型寄存器（FSR、SSR）和两个浮点寄存器（FSF、SSF）。32 位虚拟机实现中，对于大于 32 位的数值，需要将低位扩展到 SSR 或者 SSF 中。而 64 位中，由于每个寄存器都有 64 位，不需要扩展。

6. 特殊用途寄存器的实现

在 Java 虚拟机的 MIPS 64 位移植的实现中，需要保存一些信息到寄存器中，如方法对象指针、调用者 SP 值、即将解释的下一条字节码指针等。为了便于开发，将这些常用的信息进行宏定义，固定的指向某个整型寄存器。在下次使用时，即可通过宏定义进行寄存器操作，这样一方面减少了开发过程中的一些人为错误，另一方面提高了代码的可阅读性和易维护性。

2.4 模板解释器的 MIPS 64 位实现

基于模板的解释器由模板解释器、模板表、模板解释生成器三大模块构成。下面将分别介绍这些模块的功能和需要进行的移植实现。

图 2-2 给出了解释器执行的流程图。虚拟机开始从应用程序的 main 方法进入（即 Java 方法），根据当前字节码和栈顶状态在派遣表中查找对应的字节码入口地址。该入口地址指向模板表对应的本地汇编入口。在字节码执行完成时检查当前线程是否需要进入安全点。如果需要，则转入 safepoint 入口执行。执行过程中遇到异常，需要转入异常处理入口执行。如果需要调用 native 方法，则转入 native 入口地址执行等。

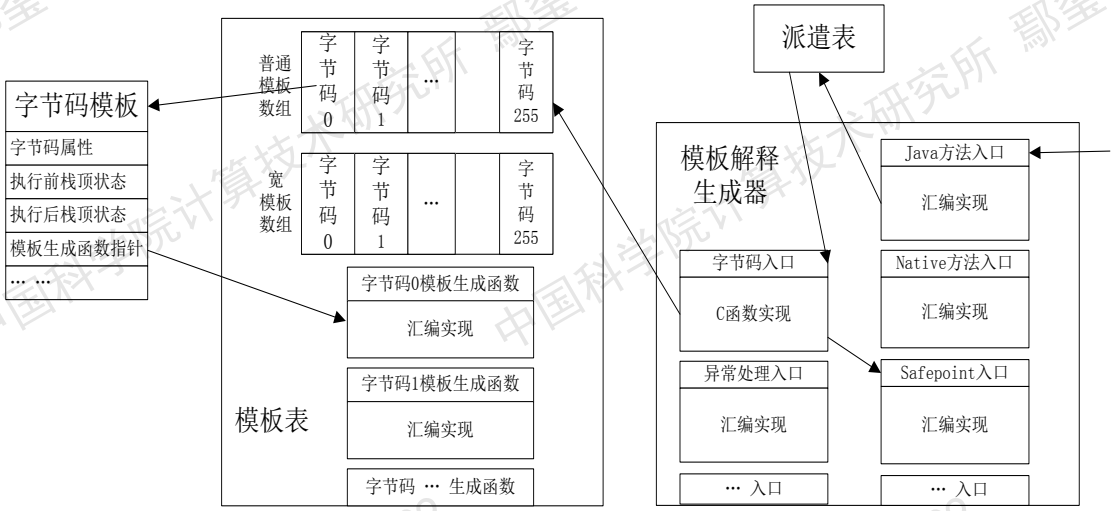


图 2- 2 模板解释器执行流程图

模板解释器定义了三个派遣表（分别是：正常派遣表、safepoint 派遣表和正在执行的派遣表）和各种入口地址指针（包括：方法入口地址、去优化入口地址、字节码入口地址等）来完成整个解释器的执行。派遣表是一个二维数组，行为栈顶状态，列为字节码编号，元素为模板入口地址。解释器根据当前栈顶状态和字节码编号从派遣表中获取字节码的模板入口地址。在虚拟机启动时初始化这些派遣表和入口地址指针。运行过程中通过转入不同的入口地址实现程序的运行。

模板表中定义了两个字节码模板数组：普通字节码模板数组和宽字节码模板数组（用于获得对更多局部变量和更大立即数的访问）。它还为每个字节码定义了对应的模板生成函数。模板生成函数用于生成每一条 Java 字节码对应的本地汇编实现。

模板解释生成器为 Java 虚拟机解释执行过程中的一些特殊处理提供通用接

口支持。如进入 Java 方法、native 方法前的栈帧建立以及参数传递，处理 Java 虚拟机解释执行过程中遇到的 safepoint、运行时异常、去优化等。同时提供了这些处理实现的入口地址。图 2-2 中列举了一些常用处理实现的入口。

Java 虚拟机的解释器的移植需要对图中所有涉及到汇编实现的地方（主要是模板表中的模板生成函数和模板解释生成器）进行移植。由于 MIPS 64 位中寄存器宽度、地址位宽的增加和算术指令、移位指令、访存指令的重新设计导致所有的汇编程序需要重新实现。为此，需要进行 MIPS 64 位解释器的整体移植。接下来的两小节将分别从模板生成函数和模板解释生成器两方面进行解释器的移植实现。

2.4.1 模板生成函数实现

由于 MIPS 64 平台地址宽度和指令的增加，几乎涉及到所有的字节码模板都需要进行移植实现。下面从如下几个方面介绍具体的移植实现：

1. 针对地址空间宽度变为 64 位所做的移植

由于 MIPS 64 位地址空间的扩展，导致 Java 虚拟机中局部变量区和常量池地址空间都相应的扩展为 64 位。所以，对于常量池和局部变量区的访问都需要移植。几乎包含 Java 字节码中除算术运算、逻辑运算指令等不涉及常量池和局部变量访问的指令外的所有指令。由于篇幅原因，下面以压入常量池中的项指令（ldc）为例进行说明，其它字节码指令的移植实现类似。

ldc 字节码的含义是通过 index 索引在当前类的常量池中查找对应的项，并将其值压入操作数栈中。index 索引通常是 8 位，如果该类的常量池超出 8 位范围，则需要一个 16 位的索引（即宽索引）。index 指向的常量池项类型必须是整数类型、浮点类型或者字符串类型中的一种。如果是整型或者浮点型，则将数值压入操作数栈。如果是字符串型，需要创建一个类实例并初始化，最后将实例指针压入操作数栈。

图 2-3 为 ldc 模板生成函数的实现流程。首先判断当前索引为普通索引还是宽索引，如果是普通索引，从字节码流中取出一个字节的索引值；如果是宽索引，则从字节码流中取出两个字节的索引，计算出索引值并将其保存在临时寄存器中。接着，获取当前类的常量池基址和标签基址并将其保存在临时寄存器中，然后根据标签基址和索引获得当前索引指向的项的类型。最后，根据不同的类型值采用不同方法得到对应的数值或者指针值。如果当前类型是未解析的类，则调用运行时方法解析当前字符串对应的类。否则，所有的值都已经保存在常量池中。根据整型、浮点型和对象采用不同的加载方式，装载到栈顶缓存寄存器中，并将对应的栈顶状态压入操作数栈中。

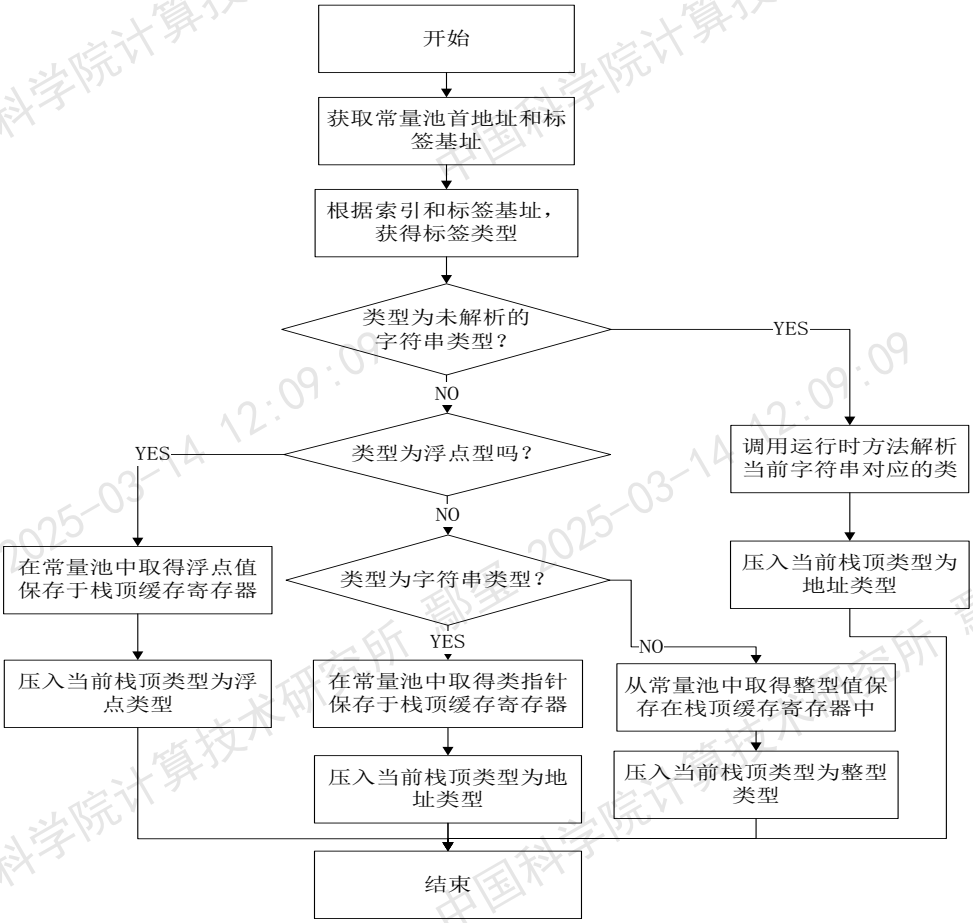


图 2- 3 ldc 模板生成函数实现流程图

64 位移植与 32 位虚拟机移植实现不同的地方。具体表现为如下几方面：

- 临时寄存器分配及浮点栈空间分配

临时寄存器相关说明在上节寄存器移植中已经给出。32 位虚拟机中浮点值在栈中分配 4 个字节，而在 64 位 Java 栈中采用 8 字节保存，低 4 个字节保存浮点值，高 4 个字节用 0 填充。

- 运算指令移植

MIPS 架构，所有的运算指令都是在寄存器上进行的，由于 MIPS 64 位寄存器宽度变为 64 位。所以所有移植中涉及到的算术运算和移位指令都需要移植。

- 索引移位移植

常量池地址空间变为 64 位，相应的，需要将索引乘以一个字的宽度。然后通过基址加偏移获得常量池中的数值。

- 加载指针移植

加载一个 32 位整型值到 64 位寄存器和加载一个指针到 64 位寄存器实现，在 MIPS 32 位中实现一致，采用一条加载指令（lw）实现。而对于 MIPS 64 位，指针为 64 位。所以需要采用 64 位加载指令（ld）实现。

2. 针对 Java 数据类型 long 型、double 型所做的移植

Java 虚拟机规定 long 型和 double 型为 64 位。原 32 位中整型寄存器和浮点寄存器都只有 32 位，而在 64 位平台下，整型寄存器和浮点寄存器宽度都扩展为 64 位。为此需要对所有与 long 型和 double 型相关的字节码模板生成实现进行 64 位移植。具体从如下几方面进行说明：

- long 型或 double 型数据存储移植

long 型采用整型寄存器存储，double 型采用浮点寄存器存储。原 32 位实现中需要两个 32 位寄存器存储的地方 MIPS 64 实现中都需要将其改造为一个寄存器单元。如 dload 字节码，将一个 double 型局部变量装载到操作数栈。32 位实现中需要将高四个字节保存在 SSF，低四个字节保存在 FSF 中；而 64 位实现中所有字节保存在 FSF 中，SSF 清零。

- long 型或 double 型运算移植

由于用于保存 long 型或者 double 型的寄存器的改变，导致所有 long 型和 double 型运算都需要移植。本节以 lmul 即 long 型乘法字节码模板生成实现为例进行说明。图 2-4 给出了 32 位虚拟机与 64 位虚拟机的不同实现。

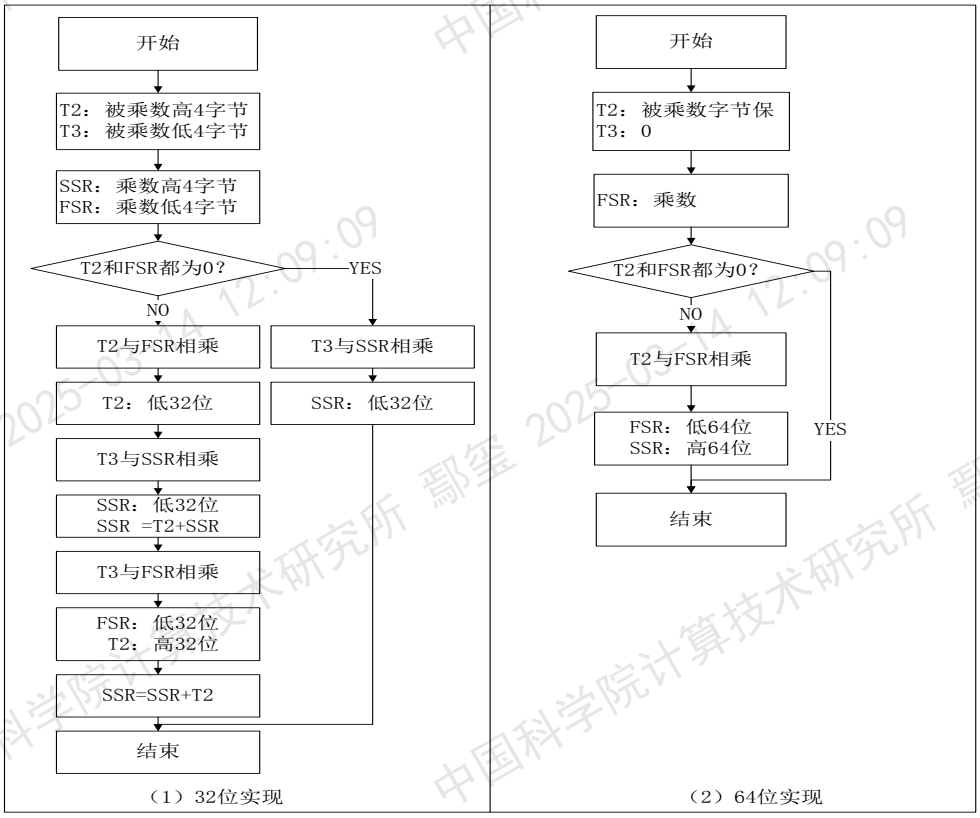


图 2- 4 lmul 模板生成函数 32 位与 64 位实现对比图

如图所示，32 位实现中乘数保存在 SSR 和 FSR 中，被乘数保存在 T2 和 T3，整个运算过程需要调用 3 次 MIPS 32 乘法指令才可以实现，代价极高。而 64 位实现中仅需要一次 MIPS 64 乘法指令即可实现，极大的提升了数值计算的性能。且 32 位 long 型乘法中只能够保存 64 位的精度，而 64 位乘法实现中可以保存 128 位的精度。提升了 long 型运算的精度。

2.4.2 模板解释生成器实现

Java 虚拟机在进入方法调用、异常处理之前，需要调用模板解释生成器进行一些特殊处理，如 Java 方法调用前需要建立 Java 栈帧、异常处理前需要清空表达式栈、native 方法调用前进行参数传递等。模板解释生成器提供了对这些特殊处理的汇编实现，同时返回一个入口地址供调用者调用。下面从三个方面介绍模板解释器的移植实现：

1. 建立解释栈帧实现

解释栈帧中保存了方法运行时所需要的方法返回地址、调用者 FP、调用者 SP、调用者上次的 SP 指针、局部变量、方法对象、方法数据、常量池指针、当前字节码指针和表达式栈底指针信息，如图 2-5 所示。其中方法数据为解释器执行过程中的剖析数据，为编译器优化提供数据。当前字节码指针用于 native 方法调用返回时查找下一条字节码。

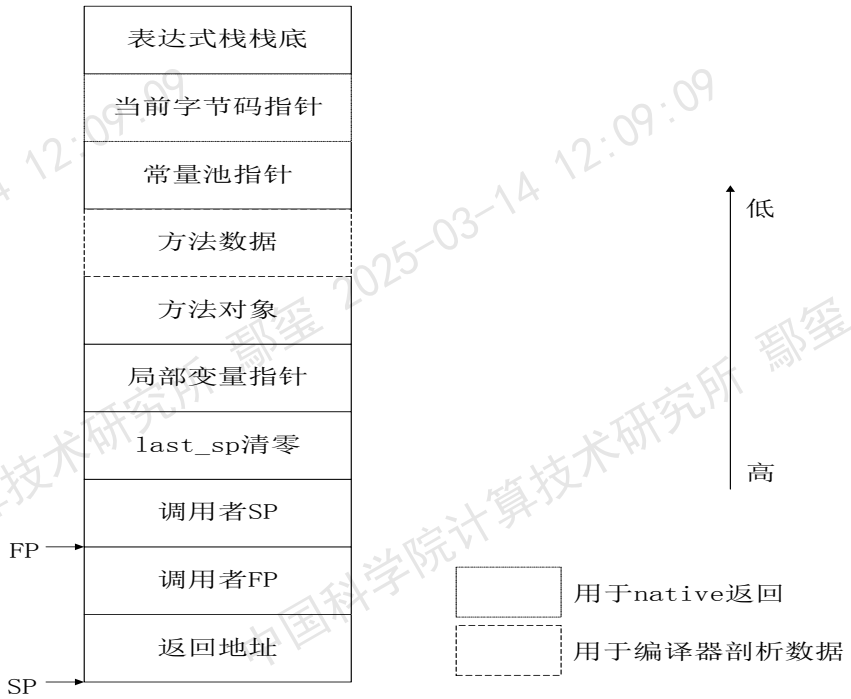


图 2- 5 解释栈帧结构图

由于 64 位数据宽度和地址空间的扩展，导致 Java 栈帧中字宽（一个字所占的字节数）发生变化。原 32 位虚拟机中每个字为 4 字节，64 位虚拟机中每个字变为 8 个字节。对解释栈帧的移植需要对栈的所有操作指令，如栈帧偏移、存储数据到解释栈帧等指令使用 MIPS 64 存储和算术指令实现。

2. 方法入口处汇编实现

方法入口处汇编实现包括 Java 方法、native 方法、safepoint 方法、异常方法、去优化等的入口处理。不同方法需要进行的具体处理实现不同。这些处理包括参数传递、栈帧建立、同步处理、目标跳转、返回值处理等。整个过程都会涉及对 Java 栈帧的访问以及寄存器存储的处理。这些都需要进行移植实现。由于篇幅原因，在此不介绍每个入口的具体处理实现流程。

3. 参数传递移植实现

这里所说的参数传递是针对本地方法（C 函数）而言的。Java 虚拟机提供了一种从 Java 程序调用 C 程序的实现机制。由于 Java 程序运行过程中的状态都保存在 Java 栈中，而 C 函数调用需要遵守 C 编译器的调用接口约定，如参数寄存器的使用、堆栈建立和参数传递等。因此，在调用 C 函数前，需要将当前 Java 栈帧转化成符合 C 编译器要求的栈帧结构。MIPS 32 本地栈帧遵循 o32 ABI 标准，而 MIPS 64 本地栈帧遵循 n64 ABI 标准。二者在参数寄存器个数、传参规则等方面的区别如表 2-3 所示。

表 2- 3 参数传递约定 o32 与 n64 对比表

不同点	o32 ABI	n64 ABI
SP 对齐位数	8 字节对齐	16 字节对齐
整型参数寄存器	整型 a0-a3 寄存器，4 个	整型 a0-a7 寄存器，8 个
浮点参数寄存器	浮点$f12-$f14 寄存器，4 个	浮点$f12-$f19 寄存器，8 个
堆栈参数空间	需要为参数寄存器预留堆栈空间	不需要在堆栈中分配空间
使用浮点寄存器传参规则	只有第一个参数是浮点型，才可用 FP 寄存器传递参数	前 8 个参数中如果出现浮点数，则在浮点寄存器中传递
整型或者指针型返回值	放在 v0 寄存器，如果是 long long（64 位整数）延伸到 v1 寄存器	放在 v0 寄存器
浮点返回值	放在$f0 寄存器中，双精度则延伸到$f1	放在$f0 寄存器中，$f2 用于返回包含两个浮点值的结构
返回值过大	如果函数返回数据结构超过两个寄存器，调用者必须在堆栈里分配一个匿名的结构变量，设置一个指针 v0 隐式指向该结构	

由于寄存器个数和栈地址空间宽度的改变以及 o32 和 n64 约定区别，导致 64 位 Java 虚拟机中需要对参数传递实现机制进行改造。

Java 虚拟机定义了十种基本类型，分别是 int、long、float、double、object、array、bool、char、byte、short。其中不够一个字节的类型如 bool、char、byte、short 向上提升为 32 位，long、double 为 64 位，剩余类型为 32 位。图 2-6 左侧 Java 栈中给出了 10 种基本类型及其所占的 Java 栈空间。并给出了在当前 Java 栈中参数区为图中左侧 11 个参数的情况下，32 位与 64 位虚拟机的传参实现。参数自下而上进行传递。

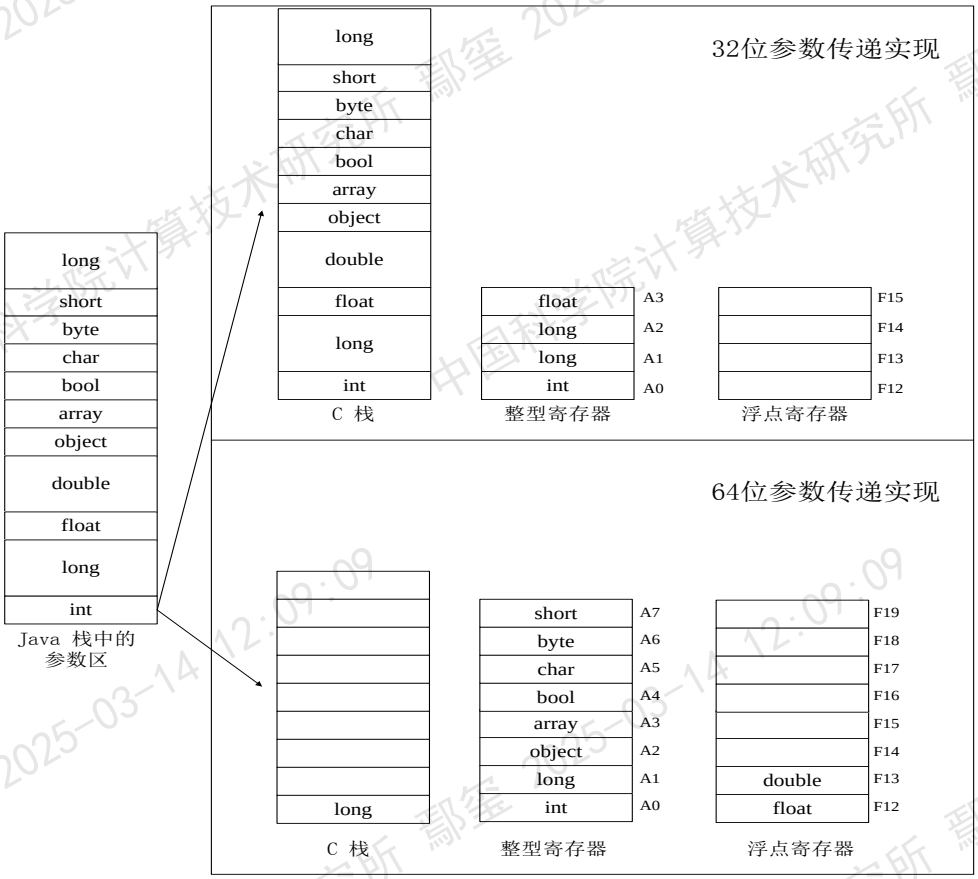


图 2- 6 32 位与 64 位参数传递实例图

32 位虚拟机中参数传递实现如图 2-6 上半部分所示。由于 MIPS o32 规定前四个参数在参数寄存器中传递，但 C 栈中依然需要保留该四个寄存器的空间。大于 4 的参数在栈中传递。因此，整个 C 参数栈区跟 Java 参数栈区相同。对于 long 型参数，需要进行特别说明。由于 32 位寄存器宽度和栈地址空间都是 32 位，所以传递一个 long 型数据需要两个整型寄存器或者两倍的栈空间。浮点类型采用跟 int 型同样的机制传递。double 型采用同 long 型同样的机制传递。

64 位虚拟中由于栈地址空间宽度为 64 位。所以，小于 64 位的类型扩展到 64 位传递，对于 long、double 型可以在一个栈空间中传递。此外，n64 约定中去除了在栈空间预留寄存器空间的约定。栈中只保留超出寄存器个数的参数。一方面节省了栈空间，另一方面也减少了栈操作。提升了参数传递的效率。

由于 MIPS 64 中参数寄存器的个数为 8 个，位宽扩展为 64 位，整型和浮点型加起来一共可以传递多达 16 个参数。除了 float、double 类型在浮点寄存器中传递外所有的类型都在整型寄存器中传递。图 2-6 下半部分给出了 64 位参数传递的实现。由此可见，64 位参数寄存器传递可以满足绝大多数应用程序的需求，极大的提高了参数传递的效率。

2.5 JIT 编译器的 MIPS 64 位实现

JIT 编译器只对调用次数超过阈值和循环执行超过阈值的方法进行编译。图 2-7 给出了 JIT 编译器的执行流程。编译器首先将字节码转换为高级中间表示，接着经过 LIR 生成器，将其转换为低级中间表示，在这个过程中进行寄存器的分配和 LIR 操作数与寄存器的映射。接着经过 LIR 汇编器将其转换为本地汇编表示执行。LIR 汇编器将 LIR 操作使用本地汇编实现。接下来分三小节从编译栈帧映射、LIR 生成器和 LIR 汇编器三方面对 JIT 编译器平台相关的部分（见图中虚线框表示）进行了实现。

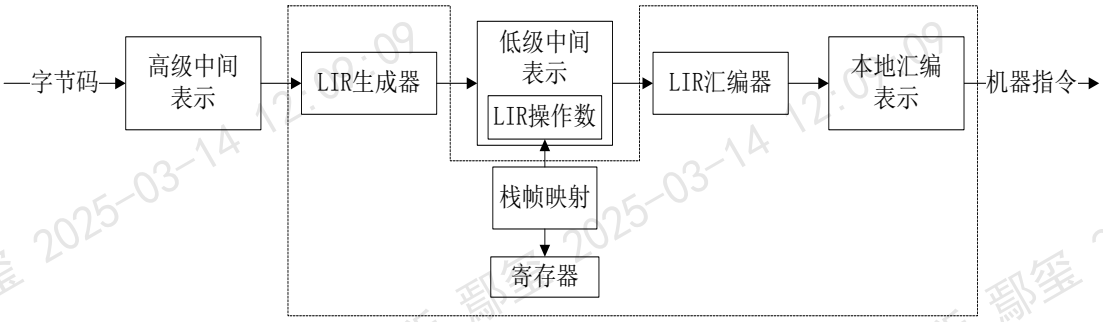


图 2- 7 JIT 编译器执行流程图

2.5.1 编译栈帧映射实现

由于低级中间表示（LIR）上的所有操作都是基于操作数类 LIR_OprDesc 的。而本地汇编实现的所有操作都是基于寄存器进行的。所以需要定义寄存器与 LIR_OprDesc 的一个对应关系，在此称为栈帧映射（FrameMap）。

LIR_OprDesc 描述了 LIR 指令操作数的数据结构。LIR 操作数有四种存储类型：指针类型、栈类型、整型寄存器类型（cpu）、浮点寄存器类型（fpu）。

每种寄存器类型中又定义了单寄存器类型（single 类型）和双寄存器类型（double 类型）。单寄存器类型是指采用一个寄存器保存的类型。双寄存器类型指采用两个寄存器分别保存数值的高位和低位的类型，一般用于如 long 型和 double 型的操作。LIR_OprDesc 由固定的 32 位表示。其位结构图如图 2-8 所示。低 3 位用于表示存储类型；3 到 6 位表示数据类型 int，long，object（对应 Java 基本类型的 T_OBJECT、T_ARRAY），pointer，float，double；7 到 8 位表示操作数大小，针对 Java 基本类型（long 和 double）的大小 double_size 和其他类型的大小 single_size；9 位表示 destroys，未使用；10 位表示最后使用位 last_use，用于线性扫描寄存器分配算法中；11 位表示是否为 is_fpu_stack_offset，只用于 X86；12 位表示是否使用 virtual 寄存器，当需要的寄存器数大于整型寄存器和浮点寄存器的和时，就采用 virtual 寄存器，virtual 寄存器也分为整型和浮点型；13 位是 X86 平台用于表示是否使用 xmm 寄存器；14 到 22 位表示寄存器 1，23 到 31 位表示寄存器 2。

图 2- 8 LIR 操作数位图

MIPS 64 位中寄存器约定的变化，导致所有映射关系都需要实现。

对于整型寄存器来说，四个临时寄存器的映射需要改为四个参数寄存器的映射。此外，在 LIR 生成器中生成汇编代码时需要用到汇编临时寄存器 AT 和堆栈寄存器 SP，由于基于栈的操作都为 64 位，所以将这两个特殊寄存器映射为双整型寄存器操作数，其他寄存器映射为单整型寄存器操作数。对于 long 型操作，32 位需要映射为两个整型寄存器，而 64 位仅需要映射为一个寄存器。

对于浮点寄存器，由于原 MIPS 32 中浮点寄存器位宽为 32 位，编译器中浮点运算为 64 位，所以 32 位编译器实现中采用成对的 32 位浮点寄存器。即 32 位虚拟机中编译器可分配的浮点寄存器个数仅有 16 个。而 MIPS 64 位中浮点寄存器的宽度扩展为 64 位，所以在 64 位虚拟机中编译器可分配的浮点寄存器个数变为 32 个。

此外，32 位虚拟机实现中栈以 4 字节对齐，由于 MIPS 64 位地址空间的扩展，所以 64 位虚拟机中采用 8 字节对齐。

2.5.2 LIR 生成器实现

LIR 生成器（LIRGenerator）用于将高级中间表示转换成低级中间表示。由于这部分涉及到操作数和寄存器的映射，所以需要进行移植。

LIR 继续保留了 HIR 的控制流结构，所以它仍然是以基本块为单位进行组织的。LIR 生成过程是将 HIR 指令转换为 LIR 操作（LIR_Op）的一个过程。最终生成的 LIR 结构如图 2-9 所示。这部分的移植使用 C 语言实现，下面以获取线程指针为例进行说明。

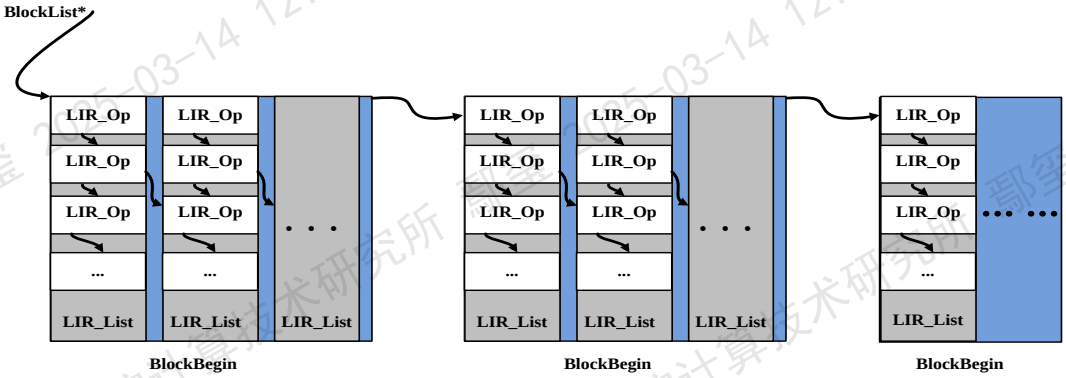


图 2- 9 低级中间表示结构图

在低级中间表示中，将很多复杂实现封装在一个 LIR 操作中，如数组拷贝、对象分配等需要很多条汇编指令实现的操作。本例中的获取线程指针就是一个例子，即获取线程指针就是一个 LIR 操作。其实现过程为，首先根据基本类型新建一个寄存器型操作数 result，在此需要说明的是，64 位中线程指针为 64 位，需要一个 T_LONG 类型的操作数，而 32 位中，指针为 32 位，需要一个 T_INT 基本类型。接着，创建一个零操作数的操作，其指令为 lir_get_thread，操作数为 result。最后返回 result 操作数。其他 LIR 生成实现类似。

2.5.3 LIR 汇编器实现

LIR 汇编器（LIR_Assembler）中定义了 LIR 操作的本地汇编实现。主要有算术操作、移位操作、数据 move 操作、监控地址操作、方法调用操作、以及如数组拷贝操作、对象分配操作等特殊操作。由于 MIPS 64 寄存器、指令等的变化，导致这些汇编实现都需要移植。下面分析算术操作的移植和数组拷贝操作的移植。其他移植实现原理相同，在此不做介绍。

1. LIR 算术操作、移位操作以及数据移动操作的移植

由于 64 位寄存器宽度的扩展以及算术指令、移位指令和存储指令的改变。导致 LIR 中的算术操作、移位操作和数据移动操作都需要移植。由于篇幅原因，下面以算术操作为例进行说明，其他操作原理类似，不一一展开。

算术操作是一个双操作数操作，为方便下面的叙述，我们将它的两个操作数分别称为左操作数和右操作数，算术操作的汇编实现是根据当前的 lir 指令对左操作数和右操作数进行相应的运算，将最终结果保存在目标操作数对应的寄存器中。下面根据不同的操作数类型以及进行的运算进行分情况论述：

- 左操作数是单整型寄存器类型

左操作数采用单个整型寄存器保存。下面根据右操作数的类型分寄存器类型和常数类型两种情况说明。

A. 右操作数是整型寄存器类型

右操作数是整型寄存器又分为单整型寄存器类型和双整型寄存器类型。如果右操作数为双寄存器类型，则从其低位寄存器中取右操作数的值。

接着，判断目标操作数类型，如果为双寄存器类型，则将接下来计算出的结果保存在其低位寄存器中。

然后，判断当前操作码，根据不同的操作码生成不同汇编指令。主要执行的运算有加法运算（lir_add 指令）、减法运算（lir_sub 指令）和乘法运算（lir_mul 指令）。下面分情况说明。

lir_add，lir_sub：加法、减法运算

在 64 位实现中，如果目标类型为整型（T_INT），则采用 32 位加法、减法指令实现。否则采用 64 位加法、减法指令实现。

lir_mul：乘法运算

在 64 位实现中，采用 64 位乘法指令（dmult，MIPS 64 指令）实现。如果类型为整型，需要将运算的最终结果进行符号扩展。

B. 右操作数是常数类型

lir_add：加法运算

如果立即数小于 16 位，则采用 64 位无符号立即数加法指令实现，否则，加载立即数到寄存器，采用 64 位加法指令实现。

lir_sub：减法运算

如果立即数小于 16 位，将立即数取反，则采用 64 位带符号立即数加法指令实现，否则，加载立即数到寄存器，采用 64 位加法指令实现。

lir_mul：乘法运算

首先将立即数加载到寄存器，其余实现同 A 中乘法实现。

- 左操作数是双整型寄存器类型

左操作数是双寄存器类型，表示该运算是 long 型运算。左、右操作数和结果寄存器都采用两个寄存器分别保存数值的高位和低位，为方便说明，分别将保存高位的寄存器叫做高位寄存器，保存低位的寄存器叫做低位寄存器。如果

目标是单整型寄存器类型，则将运算结果保存在其低位寄存器中。如果目标是双整型寄存器，在 64 位实现中，由于 64 位寄存器宽度的扩展，单个寄存器即可保存全部的运算结果，所以仍采用低位寄存器保存结果。而在 32 位中需要 2 个寄存器分别保存结果的低位和高位。

如果右操作数类型为常数，则采用临时寄存器保存其低位，高位为 0。

lir_add，lir_sub：加法、减法运算

32 位实现中：首先将左操作数的低位和右操作数的低位相加、减，保存在结果寄存器的低位；接着将左操作数的高位和右操作数的高位相加、减，保存在结构结存器的高位；最后结果寄存器的高位加上或者减去进位。

64 位实现：直接采用 MIPS 64 位加、减法指令即可实现。

lir_mul：乘法运算，请参考解释器中的 long 型乘法运算实现。

- 左操作数是栈类型或者地址类型

如果左操作数是栈类型，则从当前栈帧中获取操作数的值，如果操作数是地址类型，则从内存单元中获取操作数的值。

基于栈类型和内存类型的操作只有加减操作。根据右操作数是单整型寄存器类型或者常数类型分别采用 MIPS 64 位寄存器运算和立即数运算指令实现，并采用 MIPS 64 存储指令将最终结果保存在栈地址空间或者内存空间中。

2. 数组拷贝汇编实现

低级中间表示中加入了如数组拷贝、锁操作、对象分配等复杂操作代替了方法调用，提高了执行效率。这些实现都需要进行移植。

下面以数组拷贝操作为例说明整个移植实现。

LIR 定义了数组拷贝操作（LIR_OpArrayCopy），它定义了源数组指针操作数（src）、源位置操作数（src_pos）、目的数组指针操作数（dst）、目的数组位置操作数（dst_pos）、拷贝长度操作数（length）、默认类型操作数（default_type）和数组 CodeStub 操作数。CodeStub 对数组拷贝过程中的异常进行处理，如越界异常、类型错误异常、拷贝长度为负等。

首先，将 LIR_OpArrayCopy 中的源数组指针操作数、源位置操作数、目的数组指针操作数、目的数组位置操作数、拷贝长度操作数转换为寄存器，后面所有的操作都是基于寄存器的操作。在执行前需要将这些寄存器的值以及堆栈指针压入栈中、执行结束将这些值弹出栈，这样做是为了在运行过程中遇到安全点，保存原有状态以备恢复时使用。由于 MIPS 64 栈宽度改变，需要对压栈出栈相应的命令进行移植。

接着，根据不同的数组类型采用不同的运行时数组拷贝方法实现，如果当前数组类型未知，则调用运行时方法 arraycopy 实现，在调用前需要传递参数，

由于其需要的参数为 5 个，在 32 位实现中由于参数寄存器个数只有 4 个，所以第 5 个参数被保存在栈中，且需要为前 4 个参数预留栈空间。由于 MIPS 64 位中寄存器参数个数为 8 个，所有参数都可以直接在寄存器中传递，且不需要在栈中预留空间。如果当前类型已知，则需要判断源数组与目的数组是否为空，判断源位置、目的位置和长度是否为负，判断加上复制长度的源、目的数组是否越界，源、目的数组类型是否不匹配。如果这些条件中的任意一个成立则说明产生异常，转入 CodeStub 中进行异常处理。如果这些条件都不成立，则说明没有异常。接下来调整源、目的指针指向复制开始处，判断数组类型，如果是对象类型，调用运行时方法 oop_arraycopy 进行数组拷贝，否则，调用运行时方法 primitive_arraycopy 进行数组拷贝。由于 MIPS 64 指令集和传参机制的改变，整个过程都需要重新实现。

2.6 运行时环境实现

运行时环境定义了解释器和编译器中的一些公共实现。如异常处理、slow 数组拷贝、垃圾回收等。此外，运行时环境为解释器和编译器的相互转换提供支持。由于解释器和编译器的实现机制相差很大，如栈帧结构、字节码执行方式等，为了实现解释器和编译器的交替执行，需要一些运行机制支持，如方法调用约定和数据传送实现以及完成解释执行与编译执行转换的适配器。接下来将对这些机制和 MIPS 64 中进行的移植进行介绍。

2.6.1 运行时方法调用传参机制

本小节将从传参约定和值传递两个方面介绍方法调用相关的 MIPS 64 位移植实现。

1. 传参约定实现

编译执行过程中调用运行时方法或者本地方法时需要遵循 C 方法调用约定。从运行时方法或者本地方法返回编译执行时需要遵循 Java 方法调用约定。C 方法调用需要遵循 abi 约定，而 Java 方法调用不需要遵循 abi 约定。

Java 虚拟机中定义了一对 32 位寄存器用于参数传递，在此称之为 VM 寄存器（VMRegPair）。VM 寄存器可以为实际的物理寄存器，也可以为栈。第二个寄存器只有在传递一个 64 位数据时才会使用，如 long、double 类型。

在生成低级中间表示时，由于尚未涉及到寄存器分配，还不知道目标平台中用于参数传递的寄存器的个数，所以所有的参数都保存在 VMRegPair 中。当由低级中间表示转换为本地汇编实现时，由于平台寄存器个数已知，所以在进行参数传递前需要判断当前参数是使用寄存器传递还是使用栈传递。接下来讲

述的传参约定就是对此功能的实现。该实现需要两个数组，一个是用于保存参数类型的数组 sig_bt，另一个是参数寄存器对数组 regs。

Java 方法调用约定 MIPS 64 位实现中，整型参数寄存器使用 T0，A0-A7，浮点参数寄存器使用 F12-F19。栈每次增加两个单位（16 字节）。

针对每一个参数执行下面的实现：

- 如果参数类型为 T_VOID 类型，且其前一个参数类型不是 T_LONG 也不是 T_DOUBLE，则将相应的寄存器对设置为 bad。
- 如果参数类型为 T_BOOLEAN、T_CHAR、T_BYTE、T_SHORT、T_INT，且参数个数小于整型参数寄存器的个数，则设置 VM 寄存器对中第一个寄存器为整型参数寄存器，第二个为 bad；否则通过栈传递。
- 如果参数类型为 T_LONG、T_OBJECT、T_ARRAY、T_ADDRESS，且参数个数小于整型参数寄存器的个数，则设置 VM 寄存器对中两个寄存器分别指向参数寄存器；否则，通过栈传递。
- 如果参数类型为 T_FLOAT，且参数个数小于浮点参数寄存器的个数，则设置 VM 寄存器对中第一个寄存器为浮点参数寄存器，第二个为 bad；否则，通过栈传递。T_DOUBLE 类型与 T_FLOAT 类型类似，只是需要设置 VM 寄存器中的两个寄存器分别指向浮点寄存器或者栈。

C 方法调用约定 MIPS 64 位实现与 Java 方法类似，只是整型参数寄存器变为 A0-A7， 其它实现都相同。

2. 值传递实现

参数传递方式确定后，将进行数值传递。根据参数传递方式和参数数据类型的不同，值传递实现也不同。MIPS 64 位中根据参数数据类型实现了基本类型传递（simple_move）、浮点类型传递（float_move）、long 类型传递（long_move）、double 类型传递（double_move）和对象类型传递（object_move）。接下来以 long 类型传递为例进行说明，其他类型的传递类似。

如果原参数类型为栈类型，目标参数也为栈类型，则先将原参数值加载到临时寄存器，再将其存储在栈中。如果目标参数为寄存器类型，则直接加载原栈中的参数到目标参数寄存器中。此处需要注意的是 MIPS 32 位中由于 long 型采用两个寄存器保存，所以需要对应的存储原 VM 寄存器对中的第一个到目标 VM 寄存器的第一个，原 VM 寄存器对中的第二个存储到目标 VM 寄存器对的第二个。由于 MIPS 64 位中 long 型可直接存储在一个寄存器中，所以只需要存储一个，第二个不使用。

如果原参数类型为寄存器类型，目标参数类型为栈型，直接存储到栈中，如果目标参数类型为寄存器型，直接移动到目标寄存器中。所有这些由于 MIPS

64 位指令集的变化，都需要进行实现。

2.6.2 适配器实现

适配器实现 Java 程序的解释执行和编译执行转换。根据不同的转换需求，虚拟机实现了两种类型的适配器，分别是用于将解释执行转换为编译执行的 I2C 适配器和用于将编译执行转换为解释执行的 C2I 适配器，如图 2-10 所示。

图 2- 10 解释与编译转换适配器结构图

由于解释器和编译器参数传递规则不同，解释器的参数在栈中传递，编译器小于参数寄存器个数的部分在参数寄存器中传递，大于参数寄存器的部分在栈上传递。所以 MIPS 64 位中适配器需要实现参数的转换，获取要跳转的目标地址，最后跳转到目标地址处。I2C 适配器从解释栈帧中获取编译器入口地址。C2I 适配器从方法对象中获取解释入口地址。C2I 适配器在参数传递前需要检查当前方法是否还有未 patch 的代码，如果有，需要先执行代码 patch。再进行参数传递和目标跳转。

2.7 调试手段和问题解决实例

Java 虚拟机是一个复杂的系统软件。由于本次移植的代码规模较大，并且涉及很多深层次的原理，因此在移植过程中需要投入大量的时间进行功能调试和问题解决。

在虚拟机的调试过程中，存在以下难点：

1. 虚拟机中存在多种执行模式及转换行为。对于某一个 Java 方法，有时候是以解释模式执行，有时候又可能是以编译方式执行。在编译执行方式中，又有可能由于不同优化算法的作用而存在不同的执行路径和分支情形。

2. 动态编译器调试难度大于普通应用程序。由于动态编译器在运行过程中生成需要执行的指令，所以它不像普通程序那样可以在源代码任意位置加入打印、跟踪语句。尤其是在指令执行过程中，在寄存器、指令层面出现的信息非常难于获取。

3. 很多多线程并发行为，存在大量随机的出错现象，问题复现的周期较长，最长可能要一到两天才能复现一次，因此其调试难度远高于常规的应用软件。

即使存在上述难度，在本文的工作中仍然做到了解决所有在测试中出现的问题。下面以在实际调试过程中遇到两个典型案例为例，阐述在 Java 虚拟机调试过程中所使用的分析思路和调试手段。

2.7.1 随机多线程错误案例

- 错误现象

在测试 SPECjvm2008 中的测试程序 xml.validation 时，以 4 线程运行，有 20%的概率程序会出现崩溃现象。崩溃的时间一般都在启动的最初几秒钟之内，但错误信息不确定，有时是非法指令（SIGILL）错误，有时是无效内存引用错误（SIGSEGV），也称为段错误。

- 问题分析

通过对错误的分析，得到如下关键信息，然后对每一项逐一确认，并列出可能的原因：

1. 多线程。是否在单线程时也会出现这样的错误，为此在同样的测试条件下，将线程数设置为 1，进行测试，最终确定单线程情况下不会出现这样的错误。可以确定这是一个多线程问题。

2. 崩溃位置不定。解释执行中可以准确的知道虚拟机下次发生的动作，而编译执行由于运行过程中会加入一些优化方法造成不同的执行路径。是否这是一个编译错误。为此，在纯解释执行模式下重复了上面的测试，实验结果表明，解释模式不会出现上述错误，为此，我们将错误定为编译执行错误。

3. 崩溃发生在启动最初几秒之内。为此对程序的执行行为进行了分析，程序在执行开始前都会进行一系列的类加载、初始化等动作。可以在调试过程中着重排查这些方面。

- 调试方案

紧接着，在纯编译执行模式（-Xcomp）下，使用虚拟机的调试选项 PrintLIRWithAssembly 打印了测试程序 xml.validation 的汇编行为，发现出错位置的代码中 T9 的值是一个无效地址。跟踪 T9 的赋值位置，发现其在 NativeCall 类中的函数 set_destination_mt_safe 中赋值，该函数是多线程安全下用于修改 patch 的目标指令。patch 是 Java 虚拟机的一种类延迟解析的优化手段，未采用 patch 之前，虚拟机执行中遇到未解析的类，会停下来等待类解析完成，再继续执行。而采用 patch 优化后，虚拟机在遇到未解析的类时，先假定类已经解析完成，生成相应的指令。同时产生一段解析类并修改目标地址的代码（PatchingStub），接着继续执行。当 PatchingStub 解析完成后，修改原来生成

的指令。此处的函数 set_destination_mt_safe 就是用于修改原有指令的。

分析该函数的实现，发现最早移植时没有保证原子性，这样就可能导致一个 Java 线程正在获取该目标地址，而另一个 Java 线程正在进行修改该地址。而在 MIPS 64 位 Java 虚拟机中由于地址为 64 位，所以加载一个地址到寄存器需要 6 条指令，如图 2-11 中的前 6 条指令所示。这样就可能导致获取目标地址的线程最终得到的是一个修改前的部分地址和修改后的部分地址合成的无效地址。

该错误原因与上述收集到的错误现象刚好吻合。

1. 多线程访问同一地址错误，对应只有多线程才存在此错误。

2. patch 位置不同，最终导致生成的目标地址不同，对应崩溃位置不确定的错误现象。

3. patch 一般是在程序刚开始执行，大量加载类的时候发生。一旦进入稳定运行，不会再有任何 patch 动作。对应崩溃发生在最初几秒内的错误现象。

```
lui t9, 0x0
ori t9, t9, 0x55
dsll t9, t9, 16
ori t9, t9, 0x639b
dsll t9, t9, 16
ori t9, t9, 0xfffff000
jalr t9
sll zero, zero, 0
```

图 2- 11 出错位置代码片段

- 解决方案

在修改目标指令的实现中，先将第一条指令改成死循环指令，即 loop：b loop（即 0x1000ffff）。再修改其他后续的指令。最后，再把第一条指令写成对应的指令。这样就可以保证修改指令的时候，别的线程读到第一条指令时便被阻塞，只有当前线程修改完所有指令，最终改完第一条指令后别的线程才可以访问。相应的别的线程访问时仍然需要将当前位置填入死循环指令，执行完成再改回原有的指令。

- 测试验证

修正后，采用相同的测试条件进行了回归测试，表 2-4 给出了测试内容和测试条件以及修改前、后的测试结果对比。由表中的测试结果对比，可以看出，修改后该错误完全修正，也没有因此而引入新的错误。

表 2- 4 随机崩溃错误修改前、后测试结果对比表

测试及测试条件	修改前	修改后
xml.validation	启动 20 次，有 4、5 次崩溃	启动 40 次完全正确
SPECjvm2008	整个测试集从头到尾运行一次在最后 xml.validation 中崩溃	反复运行 5 次，每次 4 小时，均正确

2.7.2 低频率错误案例

- 错误现象

64 位虚拟机完成后，在实际的 Web 系统应用环境中，进行压力测试的时候，大约每周或者更长时间会出现启动过程中或者启动之后刚开始压力测试时虚拟机出现崩溃的现象。出错信号固定为总线错误 SIGBUS。出错时虚拟机错误日志显示 Java 方法调用栈的栈顶方法为解释运行。最后出错的 MIPS 指令的二进制为 00 00 2e 8c。这种错误出现的频率极低，在实验室内部我们采用 3 台龙芯机器同时运行 SpecJVM2008 中的测试集 compiler，指定并发 16 线程，运行一整个晚上（大约 14 小时），仅出现一次。

- 错误分析及排查过程

由于错误描述中只有在启动时或者启动后的几秒会发出崩溃，所以考虑控制测试程序的运行时间，提高问题复现频率。开始设置为 5 秒，如果程序在 5 秒后还未崩溃，强行终止，重头运行。结果发现运行 210 次，依然没有任何错误。

延长程序运行时间为 10 秒，增加测试机器数为 3 台，这样平均每分钟可测试 18 次。运行 10 分钟，大概 180 次，发现依然没有错误发生。

由于错误提示最终方法为解释执行，纯解释运行测试集，加快解释器的运行频率。此外，考虑到 web 应用压力测试为多线程环境，于是增加线程数为 8 个线程。继续运行。运行 5 分钟，大约 122 次，其中在第 8 次的时候出现错误。

由此推断，是解释器引发的问题。自此，错误可以在 5 分钟内复现一次。下一步可以在运行过程中加入调试信息进行错误调试。图 2-12 为加入调试信息后打印的错误日志片段以及错误发生时寄存器和垃圾回收的信息。

```
Registers:
R0=0x00000000,  AT=0x9e952886,  V0=0x5345bda0,  V1=0x00000000,  A0=0x72732800,  A1=0x2f20f918,  A2=0x72732800,
A3=0x72732800,  T0=0x53106570,  T1=0x52a6efe8,  T2=0x2f20f918,  T3=0x2f2894e8,  T4=0x2f170580,  T5=0x6f742f6e,
T6=0x72ebe264,  T7=0x000000c0,  S0=0x2f293180,  S1=0x71e00530,  S2=0x72ebeb00,  S3=0x00000004,  S4=0x00004206,
S5=0x00000000,  S6=0x72732800,  S7=0x72ebe2d0,  T8=0x00000000,  T9=0x00000000,  K0=0x00000000,  K1=0x00000000,
GP=0x2b910d50,  SP=0x72ebe294,  FP=0x72ebe2b4,  RA=0x2be25d4c

   0x00410860 <+16>:    sw      t8,20(t9)
   0x00410864 <+20>:    b       0x410880 <buf+48>
```

```
    0x00410868 <+24>:    nop
    0x0041086c <+28>:    sw      t2,8(t9)
    0x00410870 <+32>:    li      t5,1
    0x00410874 <+36>:    sw      t5,12(t9)
    0x00410878 <+40>:    b       0x410880 <buf+48>
    0x0041087c <+44>:    nop
    0x00410880 <+48>:    addiu   t9,t9,24
    0x00410884 <+52>:    sw      t9,-20(s8)

    0x00410888 <+56>:    lw      t5,16(t4)
    0x0041088c <+60>:    addu    at,t2,t5
    0x00410890 <+64>:    lw      t6,0(at)                                    <-- error
    0x00410894 <+68>:    beq     t6,t4,0x410914 <_GLOBAL_OFFSET_TABLE_+52>
    0x00410898 <+72>:    nop
    0x0041089c <+76>:    li      t6,20

Heap
 PSYoungGen      total 27776K, used 22475K [0x51ee0000, 0x53dd0000, 0x70ee0000)      <--年轻代
  eden space 23872K, 94% used [0x51ee0000,0x534d2e48,0x53630000)
  from space 3904K, 0% used [0x53a00000,0x53a00000,0x53dd0000)
  to   space 3904K, 0% used [0x53630000,0x53630000,0x53a00000)
 PSOldGen        total 31744K, used 0K [0x32ee0000, 0x34de0000, 0x51ee0000)      <--年老代
  object space 31744K, 0% used [0x32ee0000,0x32ee0000,0x34de0000)
 PSPermGen       total 16384K, used 4178K [0x2eee0000, 0x2fee0000, 0x32ee0000)   <--永久代
  object space 16384K, 25% used [0x2eee0000,0x2f2f4a28,0x2fee0000)

51ee0000-53dd0000 rwxp 00000000 00:00 0                                          <-- 年轻代地址空间
53dd0000-70ee0000 rwxp 00000000 00:00 0
70ee0000-70ee8000 rwxp 00000000 00:00 0
```

图 2- 12 错误日志片段

由图 2-12 中的错误代码片段追查到出错指令发生在解释器中 checkcast（检查对象是否为给定类型）字节码的模板生成函数中调用的子类型检查函数 gen_subtype_check 中。表 2-5 为虚拟机最终执行的指令和源代码的一个对应表。

表 2- 5 虚拟机实际运行指令与源码对应表

指令	源码
0x00410888 <+56>: lw t5,16(t4)	lw(T5, Rsup_klass, sizeof(oopDesc) + Klass::super_check_offset_offset_in_bytes());
0x0041088c <+60>: addu at,t2,t5	add(AT, Rsub_klass, T5);
0x00410890 <+64>: lw t6,0(at)	lw(T6, AT, 0);
0x00410894 <+68>: beq t6,t4,0x410914 <_GLOBAL_OFFSET_TABLE_+52>	Assembler::beq(T6, Rsup_klass, ok_is_subtype);
0x00410898 <+72>: nop	delayed()->nop();
0x0041089c <+76>: li t6,20	move(T6, sizeof(oopDesc) + Klass::secondary_super_cache_offset_in_bytes());

接着，对错误片段中涉及到的 T2、T4、T5 寄存器的值进行了分析。发现 T2 和 T4 的值指向永久代。对照源程序，可见 T2 对应子类（Rsub_klass）、T4

对应超类对象（Rsup_klass）、T5 对应超类中的检查偏移（super_check_offset）。分析 T2、T4 的值为永久代中的值是非常正常的。由于其在后续还需要继续使用。而 T5 的值为一个偏移量，应该是很小的值，但在此为一个非常大的值 0x6f742f6e，明显有错误。为此，打印了正确运行情况下 T5 的值，一般为 0x20、0x1c、0x14 等，这些都是很小的值。T5 的值来源于 T4，为此，猜测可能是在传入的过程中 T4 的值被破坏所致。

- 错误原因

解释器中 checkcast 的执行分为两种情形：（1）slow 路径：解释器通过读取常量池的条目得到被检查的类的信息。如果常量池条目的标记为“未初始化”，则代表被检查的类尚未加载，需要先加载目标类，将目标的指针写入常量池中的对应的条目，再将其条目的标记改为“已初始化”。当类已经加载后，即可进行常规的类兼容性检查。（2）fast 路径：如果常量池中对应目标类的条目已经被标记为“已初始化”，则代表对应的类已经加载，直接进行后续的类兼容性检查动作。

经过分析，发现最终导致错误的原因是，常量池条目标记位的修改操作没有保证多 CPU 环境下的弱内存访问一致性。

例如，在龙芯双路板（每一路 4 个 CPU）上，每一个 CPU 都是 4 发射并采用乱序指令执行方式。在此架构下，每一个 CPU 读取指令 cache 中的 4 条相邻指令，接着会根据指令操作数准备好的次序，对程序中的指令进行重新排序，从而导致对内存的实际读、写顺序会与程序中指令的读、写顺序不一致。在本例中，如果多个 Java 线程同时执行 checkcast，只有唯一一个线程会进入 slow 路径，执行类加载动作，修改常量池条目标记位；其它线程则等待类加载结束，通过 fast 路径读取常量池中标记为已初始化的类信息。

表 2- 6 多 CPU 环境下 checkcast 字节码读写线程执行对照表

写线程（1 个）	读线程（多个）
slow 路径: (1) 加载类； (2) 将已经加载的类指针写入条目； (3) 将条目的 flag 修改为“已初始化”。	fast 路径: (1) 读取条目 flag 为“已初始化”； (2) 从条目中读取类指针； (3) 使用类指针访问类的信息。

表面上看，这个功能完全正确。但是在“弱内存访问一致性”架构中，表 2-6 左侧写线程的第(2)、(3)条指令就存在重新排序的可能，从而导致在某一瞬间，常量池条目的 flag 标记变为“已初始化”，而类指针的值还没有被修改。此时，如果表 2-6 右侧的读线程访问常量池条目，就可能读到 flag 为“已初始化”，

认为应该走 fast 路径，而在常量池中取出未初始化的随机值，从而使用该随机值进行内存访问而引发 SIGBUS 错误。

- 解决方法

为了强行禁止写线程对第(2)、(3)两条指令的重新排序，在 Java 虚拟机实现中加入 WRITE_MEMORY_BARRIRE，在 MIPS 64 位指令集的实现中即为一条 sync 同步指令。使其严格遵循先写入已加载的类指针，后修改常量池标记的顺序，这样就保证了其它线程一旦读取常量池标记为“已初始化”，那么它读到的类指针值一定是有效的。

- 测试验证

修改完成后，采用相同的测试条件对其进行了回归测试，表 2-7 给出了测试内容和修改前、后的结果对比。

表 2- 7 解释器崩溃错误修改前、后测试结果对比表

测试程序	修改前	修改后
纯解释测试 compiler，8 线程三台机器	最长 5 分钟就必定会复现错误	运行 2 小时，没有发生任何错误
上述测试条件，增加运行，进行高压力测试	运行 1 晚上，至少会出 1~2 次错误	运行 2 个晚上，再没有出现任何一次错误。

由表 2-7 测试结果对比可见，错误修改后，原错误现象没再发生，也没有引入新的错误。

2.8 本章总结

本章第一部分分析了 OpenJDK 6 的整体结构，第二部分基于此结构对实现模块的相关工作进行了分析。第三部分从 MIPS 64 寄存器规范和寄存器分配策略两方面对 MIPS 64 位 Java 虚拟机的寄存器进行了实现。第四部分从模板生成函数和模板解释生成器两个方面介绍了 MIPS 64 位模板解释器的实现。第五部分分三小节介绍了 JIT 编译器的 MIPS 64 位实现，分别是：编译栈帧映射实现、LIR 生成器实现、LIR 汇编器实现。第六部分从运行时方法调用传参机制和适配器实现两个方面介绍了 MIPS 64 位运行时环境的实现。最后一部分总结了 OpenJDK 6 调试过程中的一些难点并通过随机多线程错误和低频率错误两个典型的案例进行了分析说明。

第三章 龙芯 MIPS 64 位 Java 虚拟机优化

随着虚拟机技术的逐渐成熟以及 Java 虚拟机的迅速发展，现有 OpenJDK 6 中已经投入很多项优化，并已经产生非常深入的优化效果。本章基于龙芯平台针对 MIPS 体系结构自身的特点，进一步设计和实现了若干项优化，分别是获取 Java 线程控制结构首地址优化、JNI 接口调用优化和 Safepoint 页面分配优化。这些优化实现在龙芯 3A 平台 OpenJDK 6 中有明显的性能提升效果。

本章首先讲述各项优化方法的基本原理，对于每一项优化效果的完整测试方法和实验数据在第四章中阐述。

3.1 获取 Java 线程控制结构首地址的优化方法

完成 64 位 Java 虚拟机后，采用 SPECjvm2008 测试工具对其进行性能测试，并与 32 位 Java 虚拟机性能测试结果进行对比，发现 64 位 Java 虚拟机在某些测试集上性能不如 32 位好。使用 Java 调试选项 PrintLIRWithAssembly 打印 32 位与 64 位 Java 虚拟机运行测试集的 LIR 汇编结果。经过对比，发现 get_thread 方法在 32 位虚拟机中仅需要 6 条汇编指令即可实现，而在 64 位虚拟机中却使用了 102 条指令，其指令条数为 32 位虚拟机的将近 17 倍。表 3-1 为 32 位与 64 位 Java 虚拟机中 get_thread 方法的汇编指令对比表。

表 3- 1 32 位与 64 位 Java 虚拟机中 get_thread 方法的汇编指令对比表

32 位	64 位

add t4, sp, zero
srl t4, t4, 12
sll t4, t4, 2
lui at, 0x2c10
add at, at, t4
lw t4, 0xeb0(at)
	sd at, 0xfffffff8(sp)
	daddi sp, sp, 0xfffffff8
	//此处省略 44 条汇编指令，用于保存调用者保存寄存器 v0,v1,a0,a1,a2,a3,a4,a5,a6,a7,f0,f12,f13,t0,t1,t2,t3,t8, t9, gp, ra, fp 的值
	daddiu a0, zero, 0x1 //获取线程索引值
	lui t9, 0x0
	ori t9, t9, 0x55
	dsll t9, t9, 16
	ori t9, t9, 0x5660
	dsll t9, t9, 16
	ori t9, t9, 0x1050
	jalr t9
	sll zero, zero, 0
	sd v0, 0x68(sp) //保存线程控制指针到栈中
	//此处省略 44 条汇编指令，用于恢复上述调用者保存寄存器的值
	ld at, 0x0(sp)
	daddi sp, sp, 0x8

共计 6 条指令	共计 102 条指令

get_thread 方法在虚拟机中是用于获取 Java 线程控制结构首地址的。

在 Java 虚拟机实现中，每个 Java 线程都拥有一块独立的控制结构。该控制结构保存了 Java 线程运行过程中的一些全局信息（如：当前线程状态、异常信息、虚拟机的运行结果、栈的状态等）。这些信息对于当前 Java 线程是私有的，而对于线程内部的多个函数是可以共享的。这为多线程环境下，线程内部不同函数之间传递消息提供了很好的支持。Java 虚拟机定义了一个接口类 ThreadLocalStorage 来包装线程标准库 Pthreads 中对于线程特定数据元素的实现。在此称为线程局部存储区（简称 TLS）。

在 Java 虚拟机启动时，首先会对线程局部存储区进行初始化。初始化工作是调用 pthread_key_create 函数分配用于标识线程局部存储区的键值作为当前线程索引(_thread_index)。接着虚拟机会新建一个 Java 线程（JavaThread），它会调用 pthread_setspecific 函数将线程索引与 Java 线程结构首地址指针进行绑定。当运行时程序需要使用当前线程结构首地址指针时，可以通过线程索引调用 pthread_getspecific 函数获取。

3.1.1 Java 线程控制结构首地址实现

Java 线程控制结构首地址的获取方法在 32 位虚拟机与 64 位虚拟机下采用不同的机制实现。

32 位虚拟机获取 Java 线程结构首地址的方法的实现为：建立一个由当前线程私有栈空间到线程控制结构首地址（Thread 指针）的查找表。该查找表以一个页面地址为表项，在 32 位系统中，总的可用内存为 2^{32} = 4GB。每一页所占的内存字节为 2^{12} 字节，则页表项有 $2^{32} / 2^{12} = 2^{20}$ 项。每一项存放的都是当前线程控制结构首地址指针，在 32 位虚拟机中指针所占字节为 4 字节。所以总的页表所占的内存为 4×2^{20}= 4M 字节。

龙芯 64 位虚拟机中获取 Java 线程结构首地址方法的实现最初是仿照 X86_64 的方式进行实现的。在 X86 平台 64 位系统中，总的可用内存为 2^{64}。如果按照 32 位系统那样采用查找表的方法实现，则总的页表内存大小为 $8 \times 2^{(64-12)}$=32PB 字节（1PB=2^{50}），页表将会变的极其大。所以 X86_64 系统中通过调用线程标准库 Pthreads 中的函数 pthread_getspecific()实现的。这样每次获取线程控制结构首地址都需要一次函数调用和返回。

分析，前一种方式采用查找表需要消耗一定的内存空间但是只需要通过一条访存指令和几条移位指令即可实现，速度快。后者不需要消耗内存空间，但是需要一次方法调用和返回，速度较慢。如果 get_thread 方法在虚拟机执行过程中调用频繁，则势必影响程序性能。

3.1.2 获取 Java 线程控制结构首地址方法频繁性验证

为了验证 get_thread 在虚拟机运行期间的调用频繁性。在虚拟机代码中引入计数器来统计 get_thread 方法的调用次数。在虚拟机中定义一个全局变量作为 get_thread 计数器。当虚拟机启动时初始化计数器的值为 0，每调用一次 get_thread 方法，该计数值加 1。当虚拟机退出时，打印该计数器的值。经统计，发现 Java 虚拟机执行一个简单的 HelloWorld 方法所调用 get_thread 方法的调用次数为 1072 次。可见 get_thread 在虚拟机运行期间调用确实频繁，因此对于 get_thread 方法进行优化是值得的。

3.1.3 获取 Java 线程控制结构首地址-查找表优化

分析 Java 线程运行时栈地址空间发现，其栈地址空间分配很有规律，一般都指向 0x55xxxxxxxx 之内的栈地址空间地址，图 3-1 为 Java 线程运行时的一个栈地址空间图。

```
root@Loong: # cat /proc/3042/maps
120000000-12000c000 r-xp 00000000 08:01 806785 /test/j2sdk-image-64/bin/java
120018000-12001c000 rw-p 00008000 08:01 806785 /test/j2sdk-image-64/bin/java
12001c000-1201e8000 rwxp 00000000 00:00 0 [heap]
5558318000-555833c000 r-xp 00000000 08:01 1034880 /lib64/ld-2.11.3.so
555833c000-5558344000 rw-p 00000000 00:00 0
5558344000-5558348000 r--p 00000000 00:00 0
5558348000-555834c000 rw-p 00020000 08:01 1034880 /lib64/ld-2.11.3.so
555834c000-5558350000 rw-p 00000000 00:00 0
5558350000-5558358000 rw-s 00000000 08:01 1100894 /tmp/hsperfdata_root/3042
… … … …
55a8000000-55a8024000 rw-p 00000000 00:00 0
55a8024000-55ac000000 ---p 00000000 00:00 0
ffff8d4000-ffff90c000 rwxp 00000000 00:00 0 [stack]
ffffffc000-10000000000 r-xp 00000000 00:00 0 [vdso]
```

图 3- 1 Java 线程运行时栈地址空间图

因此可见，虽然龙芯 64 位指令集定义了 64 位的地址空间，但实际在虚拟机中只使用 34 位以内的地址空间。则可以考虑参考 32 位虚拟机中 get_thread 方法的实现对 64 位虚拟机进行优化。再加上龙芯 MIPS 64 位中定义一个页面为 16K，因此这个查找表可以缩减为 $2^{34} / 2^{14}$ x 8 字节（线程指针所占的字节）= 8M 字节，在现有内存配置条件下是完全可以满足的。于是参考 32 位虚拟机的实现对 64 位 Java 虚拟机中 get_thread 方法进行查找表优化，表 3-2 给出了优化前后的代码片段对比。

表 3- 2 get_thread 方法查找表优化前、后代码片段对照表

优化前	优化后

li(A0, ThreadLocalStorage::thread_index());
call(CAST_FROM_FN_PTR(address, pthread_getspecific), relocInfo::runtime_call_type);
	li48(tmp, (long)ThreadLocalStorage::sp_map_addr());
	add(tmp, tmp, thread);
	ld_ptr(thread, tmp, 0);

采用查找表优化后，重新打印了虚拟机运行测试程序的 LIR 汇编指令，发现获取 Java 线程控制结构首地址方法的实现指令由原来的 102 条减少为现在的 18 条，如表 3-3 所示。优化后的指令条数是优化前的 1/3。此外，将整个测试程序的 LIR 汇编结果存入文件，比较文件大小发现。在 32 位虚拟机下，程序汇编代码的文件大小为 3.6MB。64 位优化前为 8.9MB。64 位优化后变为 6.6MB，比优化前减少了 2.3MB。因此可见，采用查找表优化确实减少了虚拟机执行的指令条数，提升了虚拟机的执行效率。

表 3- 3 优化前、后虚拟机执行 get_thread 方法的汇编指令对比表

优化前	优化后

sd at, 0xffffffff8(sp)
daddi sp, sp, 0xffffffff8
//此处省略 44 条汇编指令，用于保存调用者保存寄存器 v0,v1,a0,a1,a2,a3,a4,a5,a6,a7,f0,f12,f13,t0,t1,t2,t3,t8, t9, gp, ra, fp 的值
daddiu a0, zero, 0x1 //获取线程索引值
lui t9, 0x0
ori t9, t9, 0x55
dsll t9, t9, 16
ori t9, t9, 0x5660
dsll t9, t9, 16
ori t9, t9, 0x1050
jalr t9
sll zero, zero, 0
sd v0, 0x68(sp) //保存线程控制指针到栈中
//此处省略 44 条汇编指令，用于恢复上述调用者保存寄存器的值
ld at, 0x0(sp)
daddi sp, sp, 0x8
	sd at, 0xffffffff8(sp)
	daddi sp, sp, 0xffffffff8
	dadd a4, sp, zero
	dsrl a4, a4, 14
	lui at, 0xf
	ori at, at, 0xffffffff
	and a4, a4, at
	dsll a4, a4, 3
	lui at, 0x0
	ori at, at, 0x55
	dsll at, at, 16
	ori at, at, 0x5e69
	dsll at, at, 16
	ori at, at, 0x63c0
	dadd at, at, a4
	ld a4, 0x0(at)
	ld at, 0x0(sp)
	daddi sp, sp, 0x8

共计 102 条指令	共计 18 条指令

3.1.4 获取 Java 线程控制结构首地址-寄存器缓存优化

查找表优化的思路是采用内存缓冲的方式替代了方法调用。进一步思考分析，是否有比内存更快的缓存方式。于是，提出了采用寄存器进行缓存 Java 线程控制结构首地址指针的优化思路。

- 寄存器缓存优化的可行性分析

下面具体从两个方面论证其可行性：

1. Java 线程控制结构首地址固定不变。在实际 Java 虚拟机运行过程中，观察 Java 线程控制结构首地址的值。经实验证实，Java 线程控制结构首地址在 Java 线程整个生命周期内保持不变，把线程控制结构首地址指针固定的保存在一个通用寄存器中，该思路可行。

2. 有可用寄存器保存 Java 线程控制结构首地址。由于 MIPS 平台具有寄存器数量大的优势，完全可以满足这个优化的需求。

- 寄存器缓存优化实现

Java 线程控制结构首地址寄存器缓存优化的具体步骤为：

1. 选取一个寄存器作为保存 Java 线程控制结构首地址专用寄存器（又名 TREG）。MIPS 寄存器有如下几类：特殊寄存器（如，R0，SP，FP 等）、临时寄存器（T0-T9）、以及存储寄存器（S0-S7）。由于特殊寄存器具有特殊的用途，所以不被选用。如果使用临时寄存器，有可能会被编译器修改其值。在此，只能选用存储寄存器。在此选择存储寄存器中的 S6 寄存器作为线程控制结构首地址专用寄存器。

2. 确保 S6 寄存器在虚拟机的执行过程中不被修改。这涉及到虚拟机代码中两大模块的寄存器使用策略修改。一是解释器模块，解释器对于寄存器的使用采用硬编码方式，所以需要将解释器代码中原来使用 S6 的地方改为其他寄存器。二是编译器模块，编译器使用线性扫描寄存器分配算法分配寄存器，原来将所有的存储器寄存器作为可自由分配寄存器，现在需要将 S6 从可自由分配寄存器中去除。

3. 在 Java 线程创建时将 S6 赋值为线程控制结构首地址指针（即 Thread 指针），另外，为了增加对特殊情况调用 get_thread 方法的支持（如判断 S6 寄存器中线程指针的值是否与通过 get_thread 方法获取的值一致），在此增加宏定义 OPT_THREAD，如果定义了该宏，那么线程结构首地址直接从 S6 中获取，否则调用 get_thread 方法获取。

- 寄存器缓存优化效果分析

查找表优化尽管消除了函数调用的开销，但是获取线程结构首地址仍然需要多条指令来散列 SP 以形成到查找表的索引和内存访问指令。在获取 Java 线程控制结构首地址调用频繁的情况下，查找表还是会带来很大的开销。

与查找表优化相比，寄存器缓存优化仅需要一条寄存器传送指令就可以实现对线程结构首地址的访问。

综上所述，采用寄存器缓存优化极大的提高了 Java 线程控制结构首地址的获取效率。

3.1.5 获取 Java 线程控制结构首地址优化效果

采用查找表缓存和寄存器缓存的方法对 Java 线程控制结构首地址的获取实现进行优化后，对 Java 虚拟机进行了 SPECjvm2008 性能测试。具体的测试方法和测试数据以及最终效果在 4.2 节中给出。

3.2 JNI 接口调用优化

Java 虚拟机通过 JNI（Java Native Interface）标准接口[33]为 Java 方法中调用 C 方法提供支持。本节实现了一种通过编译执行调用 native 方法的机制，克服了在编译执行模式下调用 native 方法需要转换为解释执行调用 native 方法的缺陷。提高了 native 方法的调用效率。本小节首先介绍 native 方法的调用机制，接着介绍 native 方法的优化实现。

3.2.1 Native 方法的调用机制

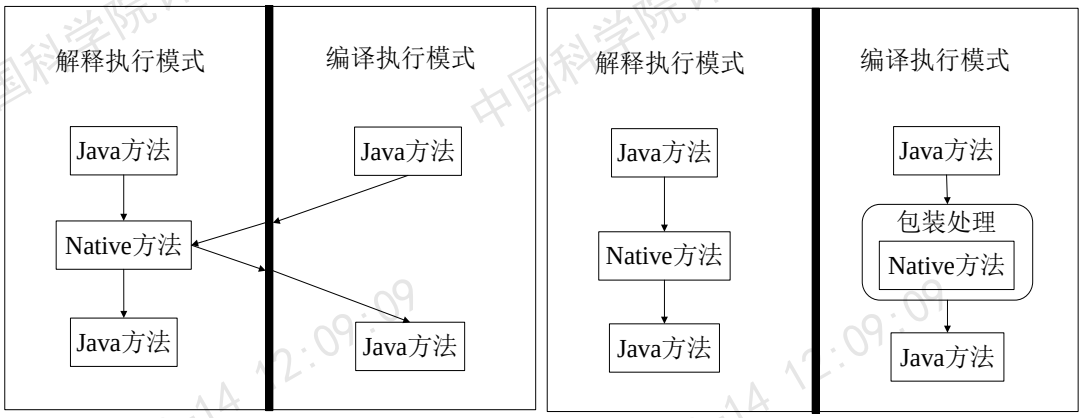
在 Java 虚拟机的实现中，对 native 方法的调用是通过模板解释器中的模板解释生成器实现的。模板解释生成器会生成一段汇编代码并将这段汇编代码的首地址返回给调用者。调用者如果需要调用 native 方法，跳转到首地址处执行。

模板解释生成器生成的这段汇编代码主要实现如下功能：

- native 方法调用前的参数传递
- C 栈帧建立与初始化
- 如果是同步方法，进行同步处理
- 执行 native 方法
- native 方法执行完成后，对返回值进行处理
- 判断当前线程是否处于安全点状态，如果是，则进入安全状态

由于模板解释生成器是解释器实现的一部分，因此无论当前 Java 方法是编译执行还是解释执行，只要调用 native 方法，就必须转换为解释执行模式，然后才能调用 native 方法，如图 3-2(1)所示。如果一个 Java 方法处在编译执行模式下，此时调用 native 方法，必须转换为解释执行模式，调用完成后再转回编译执行模式，模式切换需要 C2I 和 I2C 适配器的支持，其开销较高。

进一步分析，如果可以在编译模式下完成上述模板解释器调用 native 方法的功能，从编译执行中直接调用 native 方法，而不经过模式转换，如图 3-2(2)所示。势必可以提高 native 方法的调用效率，进而提高 Java 虚拟机性能。为此，提出了采用编译执行的方式对 native 方法调用进行优化的思路。



图（1） 优化前　　图（2） 优化后

图 3- 2 优化前、后不同执行模式下调用 native 方法对比图

3.2.2 编译执行调用 native 方法优化实现

在不改变原有虚拟机模块结构的前提下，加入一个优化开关选项。如果该开关未打开，则采用图 3-2 中优化前的方法切换模式到解释执行，调用 native 方法，调用完成切换回编译执行，继续执行原来的方法。否则，直接在编译模式下对 native 方法的调用进行包装处理。该包装处理主要包括参数调用约定处理、参数传递、特殊方法（如 static，同步）处理、执行 native 方法以及函数返回处理等步骤。

- native 包装程序实现

1. 参数传递约定转换

首先，将当前的 Java 参数约定转变为 C 参数约定。因为虚拟机在编译执行时使用 Java 方法的参数传递约定，而在 native 方法执行时需要遵循 C 方法参数传递约定。二者在参数约定方面有很大的不同，具体的约定细节见 2.6.1 节。此外，需要判断当前方法类型，如果是 static 方法，需要将参数第 2 个元素的类型设置为对象类型，用来存放当前类对象指针。

其次，确定参数存放位置以及他们所需要的栈空间大小，最终程序所需要的栈空间如图 3-3 所示。最下面 out_preserved_slots 是为具体平台本地方法实现需要预留的栈空间，在 MIPS 实现中为 0。接着是为参数预留的栈空间 vararg area 和扩展的参数栈空间。再上面是对象处理器栈空间 oopHandlearea。klass 仅在当前方法为 static 方法时，用来存放类对象引用指针所用的栈空间，通过 klass_slot_offset 获得。lock box 只有在当前方法为同步方法时才需要为其分配栈空间，通过 lock_slot_offset 获得。最上面的两个栈空间用于数据移动。

最后，拷贝参数到相应的寄存器和栈中。拷贝过程中需要根据不同的参数

类型调用不同的拷贝方法实现。具体拷贝方法在 2.6.1 节中的值传递中给出。

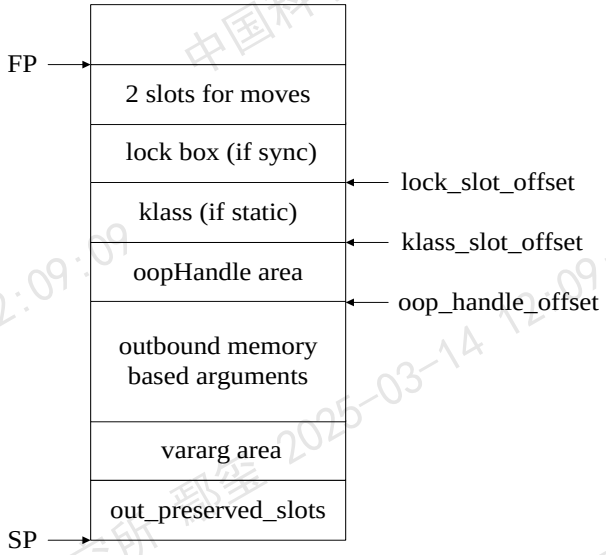


图 3- 3 本地方法调用栈结构图

2. static 方法特殊处理

当 native 方法为静态方法时，需要提前装载 static 方法对象到指定寄存器，为稍后的同步加锁处理和 native 方法调用做准备。

当 native 方法为同步方法时，需要生成 BasicLock 进行加锁处理等。

3. 调用 native 方法

在调用 native 方法前，需要依据 JNI 标准约定，将 JNI 环境变量指针（JNIEnv*）的值放入第一个参数中。在此用 A0 寄存器存储。并设置当前线程的状态为 native 状态。接着调用 native 方法。

4. 方法返回处理

Native 调用返回后，需要对返回值做一些处理，如当返回值类型为 boolean 需要转化为整型 0 或者 1 进行保存；以及对于 BYTE，SHORT 类型的扩展等。另外，还需进行 safepoint 检查，改变当前线程状态为 Java 状态，以及可能存在的异常处理，堆栈还原等。

- 优化前后实现对比

表 3-4 给出了 JNI 接口调用化前和优化后的实现对比。由表中实现可以看出优化前在进入和退出时都需要使用适配器进行执行模式切换。优化前、后参数传递实现不相同。优化前参数传递是从解释栈帧中的局部变量区传递到 C 参数寄存器或者 C 栈，而优化后参数从编译器参数寄存器（T0，A0-A7）和编译栈参数区传递到 C 参数寄存器（A0-A7）和 C 栈。在调用返回时，优化前需要恢复当前 java 字节码指针，然后跳转到返回地址执行。而优化后则直接跳转到

返回地址处。

表 3- 4 JNI 接口调用优化前、优化后实现对比

优化前	优化后
[C2I 适配器转换执行模式为解释执行] [进入模板解释生成器生成的 native 入口处] [从 Java 栈中获取参数个数] [生成固定栈帧] [计算需要在栈中保留的参数空间] [获得签名处理器 Signature Handler] [进入签名处理器中进行参数传递] [如果是 static 方法，获得类的镜像] [获得 native 方法的入口地址] [将 JNI 环境指针保存在 A0 中] [保存当前 FP 指针、PC 指针、SP 指针到栈] [改变线程状态为处于 native 执行状态] [跳转到 native 方法入口处执行] [native 执行完成，保存返回值] [改变线程状态为从 native 到 java 的转换状态] [检查安全点，进行安全处理] [改变线程状态为 java 执行状态] [清空栈中 last_sp，last_fp，last_cp 的值] [如果返回值是对象类型，将其保存在栈中] [获取当前字节码指针] [如果有异常，则处理异常] [如果有必要，进行解锁处理] [结果处理] [返回到调用点处] [I2C 适配器转换执行模式为编译模式]	[确定哪些参数需要在寄存器中保存，哪些参数需要在栈中保存，且计算出保存参数的栈空间] [检查当前内联缓存是否缺失，如果是，跳转到缓存缺失处理] [保存当前 SP 值] [从编译参数区拷贝参数到 C 参数寄存器或者栈] [如果是 static 方法，获得类的镜像] [保存当前 SP、FP、PC 值到栈中] [如果方法为同步方法，进行加锁] [获取 JNI 环境指针，并保存在 A0 寄存器] [改变当前线程状态为 native 执行状态] [执行 native 方法] [native 执行完成后，处理返回值] [改变线程状态为从 native 到 java 的转换状态] [检查安全点，进行安全处理] [改变当前线程状态为 java 执行状态] [如果有必要，进行解锁处理] [将上次的 SP、FP、PC 清零] [返回到调用点处执行]
总计 452 行代码	总计 384 行代码

由表 3-4 中的指令统计，采用 JNI 接口调用优化后，节省了 68 条指令。提高了 native 方法的执行效率。

综上所述，采用编译执行优化省去了解释编译模式切换的开销，提高了 Java 虚拟机的执行效率。

3.2.3 JNI 接口调用优化效果

采用编译执行优化 JNI 接口调用后，对 Java 虚拟机进行 SPECjvm2008 性能测试，具体的测试方法、测试数据以及最终优化效果在 4.3 节中讲述。

3.3 Safepoint 优化

Safepoint 即安全点，是 Java 虚拟机为了实现诸如垃圾回收等需要修改或者访问线程堆栈信息的机制时，必须让当前执行的除 Java 虚拟机主线程或者内存回收线程外的所有线程进入一个安全点的状态（即停止状态）。当堆栈访问或者 VM 指令操作结束时，再让所有线程退出安全状态，恢复到原来的执行状态的一种安全机制。接下来的内容将简要介绍 Java 虚拟机内 Safepoint 安全机制的实现，以及基于 MIPS 平台对于 Safepoint 页面分配实现所进行优化。

3.3.1 Safepoint 实现机制

Java 虚拟机中，定义了 3 种安全状态，分别是：不安全状态、正在进入安全状态和已经进入安全状态。

不安全状态，指的是虚拟机或者内存回收线程没有发布任何进入安全状态的指令。大多数情况下，线程都处于不安全状态。

正在进入安全状态，是一个过度状态。指的是虚拟机或者内存回收线程发布进入安全状态指令，等待所有线程进入安全状态。

已进入安全状态，指的是所有线程已进入安全状态。此时，虚拟机或者内存回收线程可以开始执行相应的指令。

Java 虚拟机中实现了一个类 SafepointSynchronize 用于管理 Safepoint。该类定义了一个 begin 方法，该方法用来通知所有的线程进入安全状态。定义了一个 end 方法，该方法用来通知所有的线程退出安全状态。

虚拟机线程或者内存回收线程就是通过调用 SafepointSynchronize 类的 begin()函数和 end()函数实现让其他线程进入和退出安全状态的。通常，Java 虚拟机中的线程在进入安全状态前会处于如下 3 种执行情形，Java 虚拟机针对不同的执行情形采用不同的机制，实现从当前状态到 safepoint 状态的转换。

- 本地执行情形

如果当前线程正在执行本地方法，则虚拟机线程不必等待 native 方法执行完成后即可提前进入安全状态，当执行 native 的线程从 native 执行返回后，判断当前虚拟机当前是否进入安全状态，如果是，则将当前线程进入安全状态。否则继续后续执行。

- 解释执行情形

Java 虚拟机中定义了三个派遣表（DispatchTable），分别是解释线程当前使用的派遣表_active_table，存放正常字节码地址的派遣表_normal_table 以及专门用于处理 safepoint 的派遣表_safept_table。派遣表中存放每个字节码对应的汇编模板入口地址。

当虚拟机进入安全状态时，用_safept_table 替换掉当前正在使用的_active_table。虚拟机从安全状态返回时，使用_normal_table 替换掉_active_table 即可实现从正常状态到安全状态的切换。

- 编译执行情形

在编译执行情况下，safepoint 机制是通过访问一个内存 polling page 页面实现的。在 Java 虚拟机启动时，会专门为 safepoint 分配一个单独的页面，并将该页面的读写方式设置为只读。

当 VM 线程调用 SafepointSynchronize::begin()让其他线程进入安全状态时，将 polling page 页面设为不可读。

当编译线程从函数或者方法块返回时，会读取 polling page 页面，如果发现不可读，就会引发异常。虚拟机在处理异常时，如果发现异常地址在 polling page 页内，会阻塞异常处理。当 VM 线程调用 SafepointSynchronize::end()让线程退出安全状态时，会设置 polling page 为可读，唤醒异常处理程序继续执行。

接下来的内容就是在编译执行模式下，对 safepoint 页面地址分配进行优化。

3.3.2 Safepoint 页面分配优化

- 问题提出

在 64 位 Java 虚拟机移植完成后，与 32 位 Java 虚拟机进行了性能对比，发现 SPECjvm2008 中的测试集 xml.validation 的测试结果 64 位比 32 位低 11.7%。多次测试稳定。通过调整内存参数、启动时间、线程个数逐个排除了垃圾回收、编译时间、多线程等外部因素引起的性能差异。最后通过打印其运行时 LIR 指令发现。32 位 Java 虚拟机在 safepoint 处理时指令条数比 64 位 Java 虚拟机少，表 3-5 为 32 位与 64 位 Java 虚拟机执行 safepoint 的汇编指令对比。

表 3- 5 32 位与 64 位 Java 虚拟机执行 safepoint 的汇编指令对比表

32 位	64 位

safepoint [t2|I] [bci:17]
lui t2, 0x2acc
lw at, 0x100(t2)
 	
branch [LT] [t0|I] [int:900000000|I] [B1]
lui at, 0x35a5
addiu at, at, 0xffffe900
slt at, t0, at
bne at, zero, 0x2c7d9cf0
nop
	safepoint [t0|I] [bci:17]
	lui t0, 0x0
	ori t0, t0, 0x55
	dsll t0, t0, 16
	ori t0, t0, 0x5ffc
	dsll t0, t0, 16
	ori t0, t0, 0x100
	lw at, 0x0(t0)
	branch [LT] [a4|I] [int:900000000|I] [B1]
	lui at, 0x35a4
	ori at, at, 0xffffe900
	slt at, a4, at
	bne at, zero, 0x0000005562120d6c
	nop

由表可见，在 32 位 Java 虚拟机下 safepoint 仅需要 2 条指令即可实现，而 64 位 Java 虚拟机下需要 7 条才能实现。我们专门把这几条指令列出来分别做 10 亿次循环执行，最后对比执行时间。发现 64 位 Java 虚拟机比 32 位 Java 虚拟机差将近 75%。据此分析，如果能够优化 64 位 Java 虚拟机中 safepoint 的实现，理论上可以提升 64 位 Java 虚拟机的性能。

- 优化前实现

在 64 位 Java 虚拟机的实现中，polling page 页面的分配采用 mmap()系统调用实现，如表 3-7 中的左侧所示。它起始地址设置为空，由系统决定映射区的起始地址。因此，polling page 页面可能是 64 位地址空间中的任意页面。

当线程访问 polling page 页面时，需要先加载 64 位的页面地址到寄存器，接着再通过一条访存指令实现。加载 64 位的页面地址采用伪指令 li64 实现，该指令需要 6 条 MIPS64 位指令实现。详见表 3-6 中的第一项。

- 优化思路一

在 Java 虚拟机运行时，观察 Java 虚拟机运行时地址空间发现，它使用的地址空间一般都是 0x00000055xxxxxxxx 之内的地址，即低 48 位地址之内。经过多次虚拟机运行实践证明，高 16 位地址空间未使用，不存在特殊情况。为此，可以对立即数加载指令实现进行优化。优化方法为，只加载低 48 位的地址，高 16 位地址填充为 0。表 3-6 中的第二项给出了优化后立即数的加载实现。

由表可见，48 位立即数加载比 64 位立即数加载指令少用了两条 MIPS 指令，减少了开销。

表 3- 6 加载立即数优化前、后实现对比表

优化前（li64）实现	优化后（li48）实现

void MacroAssembler::li64(Register rd, long imm) {
lui(rd, imm >> 48);
ori(rd, rd, split_low(imm >> 32));
dsll(rd, rd, 16);
ori(rd, rd, split_low(imm >> 16));
dsll(rd, rd, 16);
ori(rd, rd, split_low(imm));
}
	void MacroAssembler::li48(Register rd, long imm) {
	lui(rd, imm >> 32);
	ori(rd, rd, split_low(imm >> 16));
	dsll(rd, rd, 16);
	ori(rd, rd, split_low(imm));
	}

- 优化思路二

经过进一步思考，发现如果能够将 polling page 页面分配在低 32 位地址空间范围内，即可实现同 32 位 Java 虚拟机访问 polling page 页面一样的效果。即可消除 64 位虚拟机 safepoint 页面访问开销问题。

对 mmap 内存分配系统调用进行研究，发现其起始地址可以进行指定。于是，在 64 位 Java 虚拟机的 polling page 页面分配实现中，将其起始分配地址指定为低 32 位地址空间的一个地址，如 0x400000，该起始地址必须是内存分页

后每页的开始地址。经过多次实验表明，优化后，Java 虚拟机正确运行，指定该起始地址目前并未对 Java 虚拟机的返回值造成任何影响。但不排除以后可能会有一定的影响，为此，后续工作将对虚拟机运行过程中的内存空间进行探测，分配空闲的低 32 位地址空间用于 safepoint 页面地址。表 3-7 给出了优化前后 safepoint 页面地址分配的实现比较。

表 3- 7 Safepoint 页面分配优化前、后实现对比表

优化前	优化后
address polling_page = (address) ::mmap(NULL, Linux::page_size(), PROT_READ, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);	void * p = (void *)0x400000; address polling_page = (address) ::mmap(p, Linux::page_size(), PROT_READ, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);

采用低 32 位页面地址分配优化后，访问 polling page 页面地址可以直接通过一条加载指令和一条访存指令实现。采用页面地址分配优化比采用 li48 指令优化节省了 3 条 MIPS 指令。整个 safepoint 机制的实现跟 32 位虚拟机下完全相同。消除了 64 位虚拟机下访问 polling page 页面低于 32 位的性能差。

3.3.3 Safepoint 页面分配优化效果

采用低 32 位页面地址分配对 Safepoint 实现进行优化后，对 Java 虚拟机进行 SPECjvm2008 性能测试，具体的测试方法、测试数据以及最终优化效果在 4.4 节进行介绍。

3.4 本章总结

本章基于龙芯平台在已完成的 MIPS 64 位 Java 虚拟机上实现了三项优化。第一小节介绍了 Java 线程控制结构首地址获取优化实现。首先给出了 32 位与 64 位 Java 线程控制结构首地址的两种不同实现方法。接着提出 MIPS 64 位实现中可以采用与 32 位虚拟机相同的实现机制，并进行了论证。接着验证了获取 Java 线程结构首地址的调用频繁性。实现了 64 位虚拟机查找表优化线程控制结构首地址方法。紧接着提出了进一步的优化思路-寄存器缓存优化，并进行了可行性论证和具体实现。第二小节实现了 Native 方法的编译执行优化。首先介绍了优化前 native 方法的调用机制，接着提出了优化思路并实现了编译执行下的 native 方法调用，提高了 native 方法调用的效率。第三小节实现了 Safepoint 页面分配优化。开篇介绍了 Safepoint 三种执行情况下的实现机制，接着提出了编译执行情况下，Safepoin 低 32 位地址空间分配的优化思路并进行了实现，降低了访问页面地址的指令条数，提高了 Safepoint 页面地址访问效率。

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

第四章 MIPS 64 位 Java 虚拟机性能评测

本章对上一章所分析的优化方法进行性能测试和验证。本章第一小节介绍了测试本文移植的 OpenJDK 6 的软硬件环境和测试工具。接下来三小节对本文第三章分析的三项优化进行了性能测试和结果分析。最后，分别采用 1GB 内存和 3GB 内存测试了 OpenJDK 6 的性能并与 32 位 Java 虚拟机进行了性能对比。

4.1 测试平台及工具介绍

本文在龙芯 3A 单路开发板上进行了 OpenJDK 6 的性能测试。其主要配置如下：龙芯 3A 高性能通用四核（主频 800MHz-1GHz）、AMD RS780E 北桥芯片、AMD SB710 南桥芯片、四根 DDR3 插槽、板载 Realtek 8111DL 10/100/1000Mbps 网络控制器、板载 128MB 显存、4Mb Flash ROM 用于 BIOS 存储。采用的操作系统是 Debain 系统。

本文采用标准性能评估机构(Standard Performance Evaluation Corporation，简称 SPEC)提供的 Java 虚拟机性能测试工具集 SPECjvm2008 作为主要的测试工具。SPECjvm2008 测试工具集介绍如表 4-1 所示。

表 4- 1 SPECjvm2008 测试集功能表

测试名称	功能描述
compiler.compiler	测试普通 java 编译
compiler.sunflow	测试 sunflow 图像渲染
compress	测试压缩
crypto.aes	测试 AES/DES 加解密算法
crypto.rsa	测试 RSA 公钥加解密算法
crypto.signverify	测试 MD5 加密算法， 基于 RSA，DSA 的安全哈希算法 SHA
derby	测试数据库相关逻辑，包括数据库锁，BigDecimal 计算等
mpegaudio	测试 mpeg 音频解码
scimark.fft.large	测试快速傅立叶变换，使用 32M 大数据集
scimark.lu.large	测试 LU 分解，使用 32M 大数据集
scimark.sor.large	测试 jacobi 逐次超松弛迭代法，使用 32M 大数据集
scimark.sparse.large	测试稀疏矩阵乘积，使用 32M 大数据集
scimark.fft.small	测试快速傅立叶变换，使用 512K 小数据集
scimark.lu.small	测试 LU 分解，使用 512KB 小数据集
scimark.sor.small	测试 jacobi 逐次超松弛迭代法，使用 512KB 小数据集
scimark.sparse.small	测试稀疏矩阵乘积，使用 512KB 小数据集
scimark.monte_carlo	测试蒙特卡罗算法
serial	测试 socket 传输 java 序列化对象到对端反序列化
sunflow	测试 sunflow 多线程迭代测试图片渲染

xml.transform	测试 xml 转换
xml.validation	测试 xml schema 验证

4.2 获取 Java 线程控制结构首地址优化效果

本小节首先测试了 OpenJDK 6 在运行 SPECjvm2008 测试集时获取 Java 线程结构首地址方法（get_thread）的调用次数。接着测试了采用查找表优化的效果，计算出优化后性能提升幅度，并对优化结果进行了分析。最后，测试和分析了寄存器缓存优化的效果。

4.2.1 获取 Java 线程控制结构首地址方法调用次数统计

本节采用 3.1.2 中的方法，对 64 位 Java 虚拟机在运行 SPECjvm2008 中的测试集时调用获取 Java 线程控制结构首地址方法的次数做了统计。图 4-1 为方法调用次数统计结果，横坐标为 SPECjvm2008 测试项目，纵坐标为获取 Java 线程控制结构首地址方法的调用次数。由图可知，在测试集 xml.transfrom 中获取 Java 线程控制结构首地址的调用次数达到 7000，几乎每个测试集的 Java 线程控制结构首地址调用次数都超过了 2000 次。平均调用次数为 3420 次。因此可见，本文对于获取 Java 线程控制结构首地址的优化是值得的。

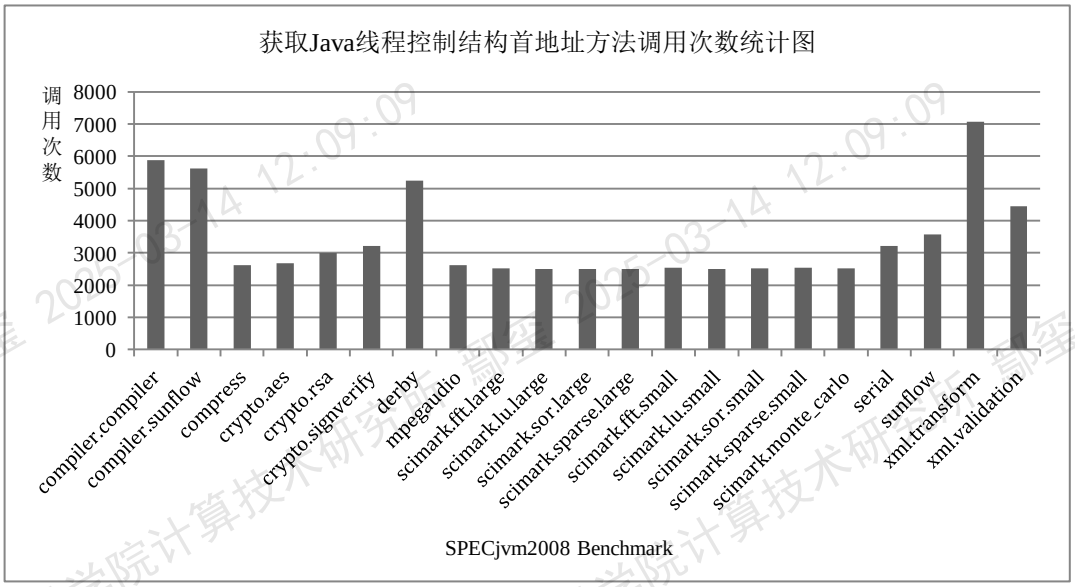


图 4- 1 获取 Java 线程控制结构首地址方法调用次数统计图

4.2.2 获取 Java 线程控制结构首地址-查找表优化效果

采用查找表的方法优化获取 Java 线程控制结构首地址的实现后，对 64 位 Java 虚拟机进行了 SPECjvm2008 性能测试，表 4-2 列出了优化前、查找表优化、寄存器优化的 SPECjvm2008 测试数据，以及两项优化的性能提升幅度。由表可见采用查找表优化后，测试集 sunflow 性能提升幅度最大，优化后提升了 166.67%。平均性能提升 13.7%。

表 4- 2 Java 线程控制结构首地址-查找表优化前、后测试数据对比表

（SPECjvm2008，-Xms1024M，-Xmx1024M）

测试集	优化前（ops/m）	查找表优化（ops/m）	性能提升(百分比)	寄存器优化（ops/m）	性能提升(百分比)
compiler.compiler	2.58	3.26	26.36%	3.31	1.53%
compiler.sunflow	0.93	1.15	23.66%	1.17	1.74%
compress	16.17	16.16	-0.06%	16.14	-0.12%
crypto.aes	0.61	0.71	16.39%	0.71	0.00%
crypto.rsa	4.64	4.79	3.23%	4.80	0.21%
crypto.signverify	6.3	7.91	25.56%	8.01	1.26%
derby	0.57	0.8	40.35%	0.85	6.25%
mpegaudio	1.09	1.19	9.17%	1.19	0.00%
scimark.fft.large	1.75	1.91	9.14%	1.92	0.52%
scimark.lu.large	0.26	0.26	0.00%	0.26	0.00%
scimark.sor.large	1.24	1.27	2.42%	1.28	0.79%
scimark.sparse.large	0.79	0.79	0.00%	0.81	2.53%
scimark.fft.small	9	12.65	40.56%	12.73	0.63%
scimark.lu.small	5.68	6.43	13.20%	6.45	0.31%
scimark.sor.small	11.71	12.19	4.10%	12.21	0.16%
scimark.sparse.small	5.35	5.31	-0.75%	5.38	1.32%
scimark.monte_carlo	0.2	0.31	55.00%	0.35	12.90%
serial	0.39	0.55	41.03%	0.55	0.00%
sunflow	0.24	0.64	166.67%	0.66	3.13%
xml.transform	1.39	1.95	40.29%	2.03	4.10%
xml.validation	2.35	3.02	28.51%	3.13	3.64%

图 4-2 为采用查找表对 MIPS 64 位 Java 虚拟机中的 Java 线程控制结构首地址进行优化后，采用 SPECjvm2008 对其进行性能测试，然后与优化之前的性能进行对比得到的效果图。图中横轴为 SPECjvm2008 的 21 项 Benchmark 测试集，纵轴为每分钟操作数，操作数越大表示性能越好。

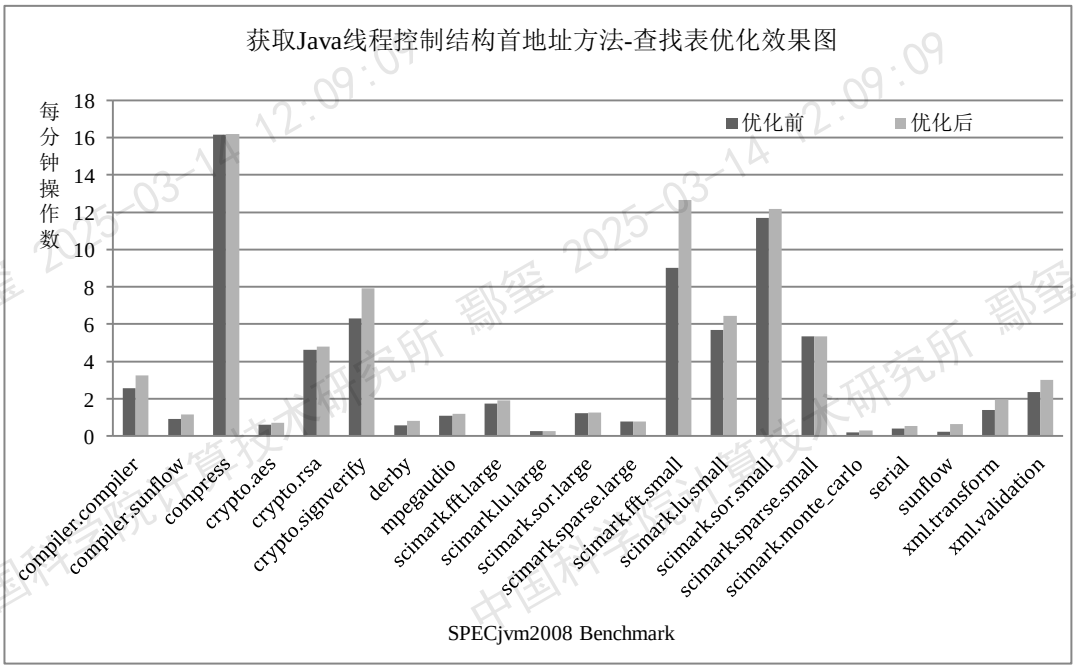


图 4- 2 获取 Java 线程控制结构首地址方法-查找表优化效果图

由图 4-2 可见，几乎每个测试集性能都有很大的提升，其中几个测试集，如 crypto.signverify、scimark.fft.small、xml.transform 等经过查找表优化后性能显著提升。结合图 4-1 中对于获取 Java 线程控制结构首地址方法的调用次数的统计可知，它们对于 Java 线程结构首地址方法的调用次数比较多，刚好验证了本优化的效果。而对于调用次数不多的测试集，如 compress、scimark.sparse.small 等性能提升不明显。

4.2.3 获取 Java 线程控制结构首地址-寄存器缓存优化效果

采用寄存器缓存的方法对 MIPS 64 位 Java 虚拟机中的获取 Java 线程控制结构首地址进行优化后，对虚拟机进行了 SPECjvm2008 性能测试，表 4-2 中后两项列出了优化前、优化后的 SPECjvm2008 中的 21 项测试集的每分钟操作数，由表可知，经过优化后，在查找表优化的基础上每个测试集性能都有所提升。其中测试集 scimark.monte_carlo 性能提升最高，优化后提升了 12.9%。

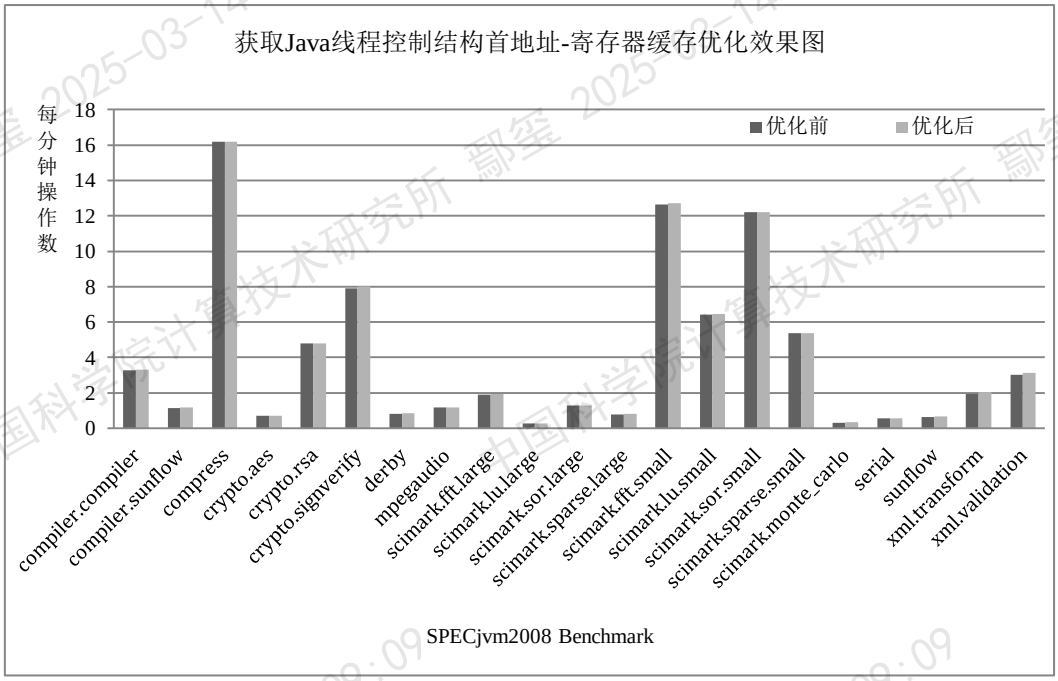


图 4- 3 获取 Java 线程控制结构首地址-寄存器缓存优化效果图

图 4-3 为采用寄存器缓存对 MIPS 64 位 Java 虚拟机中的 Java 线程控制结构首地址进行优化后，采用 SPECjvm2008 对其进行性能测试，然后与优化之前的性能进行对比得到的优化效果图。图中横轴为 SPECjvm2008 的 21 项 Benchmark 测试集，纵轴为每分钟操作数，操作数越大表示性能越好。结合图 4-1 对获取 Java 线程控制结构首地址方法的调用次数的统计分析可见，对于调用次数多的测试集，如 xml.transform、xml.validation、derby、scimark.monte_carlo 等性能提升显著。

4.3 JNI 接口调用优化效果

采用编译执行调用本地方法对 MIPS 64 位 Java 虚拟机进行优化后，对其进行了 SPECjvm2008 性能测试，表 4-3 列出了优化前后的性能数据以及性能提升百分比。由表可知，优化以后，性能最高有 59.5%的提升。平均性能提升幅度为 9.1%。

表 4- 3 JNI 接口调用优化前、后测试数据对比表

（SPECjvm2008，-Xms1024M，-Xmx1024M）

测试集	优化前（ops/m）	优化后（ops/m）	性能提升(百分比)
compiler.compiler	8.37	9.32	11.35%
compiler.sunflow	3.44	3.82	11.05%
compress	16.29	16.21	-0.49%
crypto.aes	4.11	4.43	7.79%
crypto.rsa	6.37	6.37	0.00%
crypto.signverify	12.69	13.24	4.33%
derby	3.27	3.62	10.70%
mpegaudio	6.59	7	6.22%
scimark.fft.large	1.83	1.89	3.28%
scimark.lu.large	0.26	0.26	0.00%
scimark.sor.large	1.24	1.33	7.26%
scimark.sparse.large	0.78	0.82	5.13%
scimark.fft.small	18.24	18.72	2.63%
scimark.lu.small	8.41	8.4	-0.12%
scimark.sor.small	14.12	14.38	1.84%
scimark.sparse.small	5.23	5.33	1.91%
scimark.monte_carlo	4.31	4.43	2.78%
serial	2.49	3.95	58.63%
sunflow	3.04	4.85	59.54%
xml.transform	6.58	9.43	43.31%
xml.validation	8.91	11.2	25.70%

图 4-4 给出了 JNI 接口调用优化前、后 SPECjvm2008 性能对比效果图。图中横轴为 SPECjvm2008 中的 21 个测试集。纵轴为每分钟执行操作数。数值越大表示其性能越好。

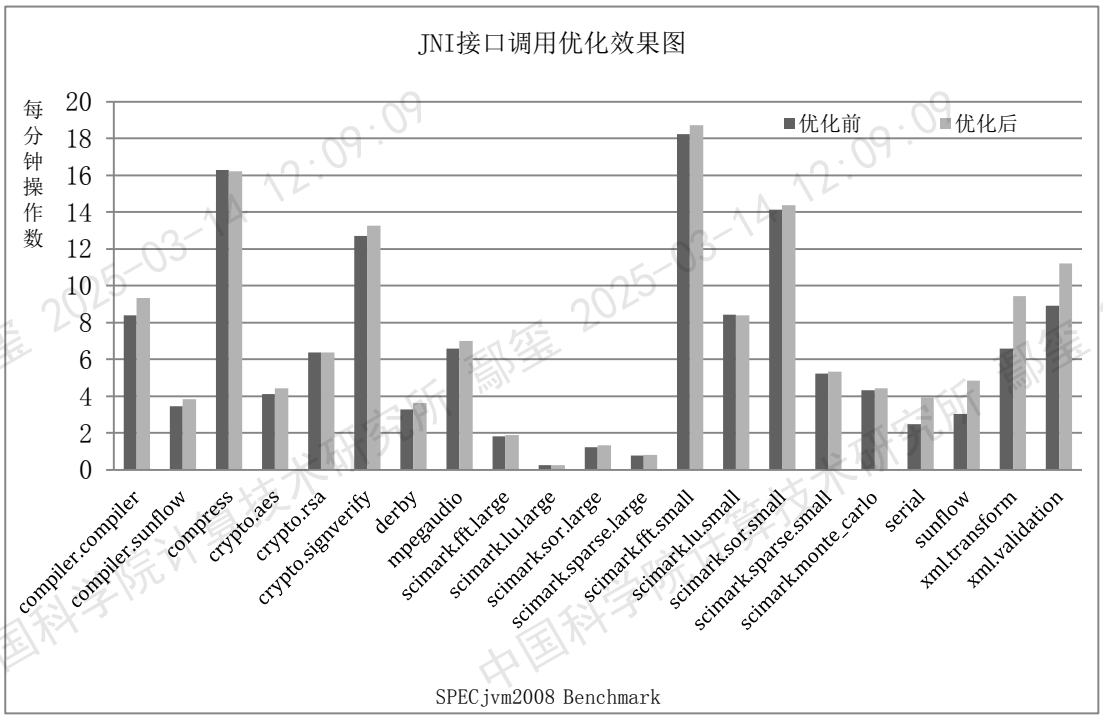


图 4- 4 JNI 接口调用优化效果图

由图 4-4 可见，SPECjvm2008 测试集中如 serial、compiler.compiler、sunflow 等测试集性能提升显著，经过分析这些测试集代码发现，这些测试集中 native 方法调用较多，所以经过 native 方法的编译优化后，性能大幅度提升。而对于某些测试集如 compress、scimark.lu.small 等测试集，经过 native 方法调用优化后性能提升不明显，这主要是由于这些测试集大都是做一些运算和算法实现，native 方法调用不多，所以经过优化后性能提升幅度不大。

4.4 Safepoint 页面分配优化效果

采用低 32 位地址分配对 MIPS 64 位 Java 虚拟机中的 safepoint 页面地址进行优化后，对 Java 虚拟机进行了 SPECjvm2008 性能测试，表 4-4 列出了优化前、优化后 SPECjvm2008 的性能测试数据以及性能提升百分比。由表中数据可知，优化后，性能最高提升 11.8%。平均性能提升 2.26%。

表 4- 4 Safepoint 页面分配优化前、后测试数据对比表

（SPECjvm2008，-Xms1024M，-Xmx1024M）

测试集	优化前（ops/m）	优化后（ops/m）	性能提升(百分比)
compiler.compiler	9.32	9.45	1.35%
compiler.sunflow	3.82	3.82	0.00%
compress	16.21	16.22	0.06%
crypto.aes	4.43	4.42	-0.23%
crypto.rsa	6.37	6.44	1.15%
crypto.signverify	13.24	13.52	2.08%
derby	3.62	3.69	1.90%
mpegaudio	7	7.04	0.57%
scimark.fft.large	1.89	1.9	0.50%
scimark.lu.large	0.26	0.26	0.00%
scimark.sor.large	1.33	1.34	0.75%
scimark.sparse.large	0.82	0.84	2.30%
scimark.fft.small	18.72	20.93	11.81%
scimark.lu.small	8.4	8.42	0.24%
scimark.sor.small	14.38	14.44	0.42%
scimark.sparse.small	5.33	5.29	-0.71%
scimark.monte_carlo	4.43	4.5	1.58%
serial	3.95	4.17	5.48%
sunflow	4.85	4.82	-0.62%
xml.transform	9.43	9.45	0.21%
xml.validation	11.2	11.42	1.92%

图 4-5 给出了 safepoint 页面地址优化前、后 SPECjvm2008 性能对比效果图。图中横轴为 SPECjvm2008 中的 21 个测试集。纵轴为每分钟执行操作数。数值越大表示其性能越好。

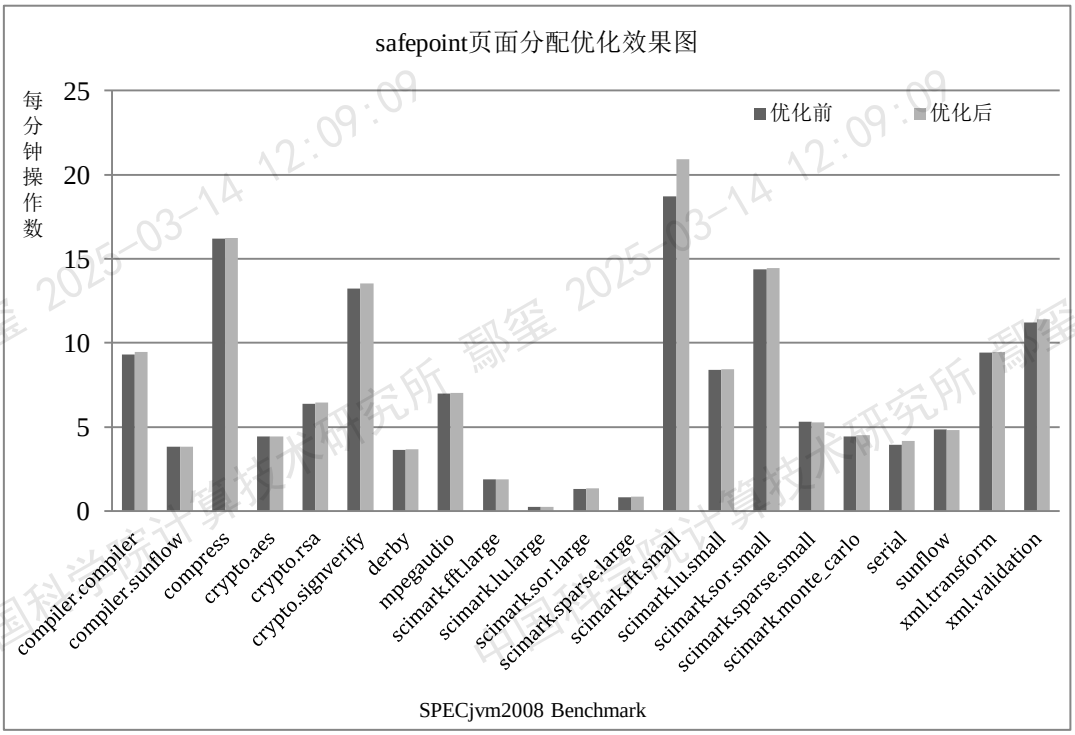


图 4- 5 Safepoint 页面分配优化效果图

由图 4-5 可见，经过 safepoint 页面地址分配优化后，SPECjvm2008 中的测试集，如 scimark.fft.small、serial、crypto.signverify 等性能提升显著。通过分析虚拟机运行过程中的行为发现，这些测试集运行过程中垃圾回收次数比较多，对应的进入和退出安全点的执行次数也比较多。所以经过 safepoint 优化后，这些测试集的性能提升幅度比较大。其他测试集性能提升不高，是由于其运行过程中进入和退出安全点比较少，所以导致优化后性能提升不明显。

4.5 三项优化综合效果

三项优化完成后，采用 SPECjvm2008 对优化前、后 Java 虚拟机的性能进行了测试。表 4-5 列出了优化前、优化后 SPECjvm2008 的性能测试数据以及性能提升百分比。由表中数据可知，优化后，平均性能提升 27.97%。

表 4- 5 三项优化前、后测试数据对比表

（SPECjvm2008，-Xms1024M，-Xmx1024M）

测试集	优化前（ops/m）	优化后（ops/m）	性能提升(百分比)
compiler.compiler	6.69	9.49	41.85%
compiler.sunflow	2.77	3.83	38.27%
compress	16.21	16.23	0.12%
crypto.aes	3.65	4.44	21.64%
crypto.rsa	7.5	7.91	5.47%
crypto.signverify	11.01	14.06	27.70%
derby	2.33	3.69	58.37%
mpegaudio	6.39	7.08	10.80%
scimark.fft.large	1.84	2.01	9.24%
scimark.lu.large	0.26	0.26	0.00%
scimark.sor.large	1.3	1.34	3.08%
scimark.sparse.large	0.83	0.85	2.41%
scimark.fft.small	14.6	22.3	52.74%
scimark.lu.small	7.58	8.46	11.61%
scimark.sor.small	13.63	14.4	5.65%
scimark.sparse.small	5.42	5.53	2.03%
scimark.monte_carlo	3.07	4.94	60.91%
serial	1.96	4.17	112.76%
sunflow	1.68	4.88	190.48%
xml.transform	5.16	9.48	83.72%
xml.validation	8.72	11.54	32.34%

图 4-6 给出了三项优化前、后 SPECjvm2008 性能对比效果图。图中横轴为 SPECjvm2008 中的 21 个测试集。纵轴为每分钟执行操作数。数值越大表示其性能越好。

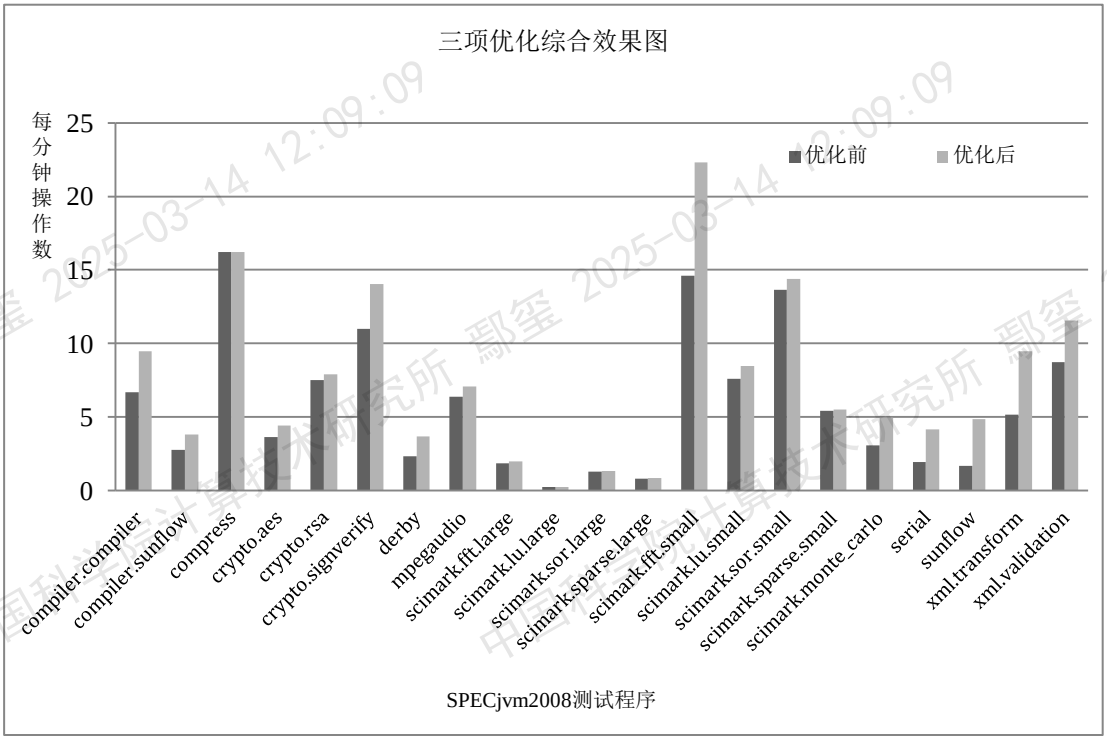


图 4- 6 三项优化综合效果图

由图 4-6 可见，经过三项综合优化后，SPECjvm2008 中的每一项测试集的性能都有所提升，其中如 serial、sunflow、xml.validation 等测试集性能提升幅度最大，主要是由于这些测试集中本地方法调用和内存访问频繁，所以经过本文中的优化后，性能大幅提升。其他测试集性能提升幅度不大，是由于测试集偏向于运算比较多，所以经过本文的三项优化后，性能提升幅度不大。但总体性能表明，本文中进行的三项优化效果显著，达到了预期目的。

4.6 64 位虚拟机与 32 位虚拟机性能对比与分析

综合了本文所有的优化后，采用 SPECjvm2008 对 32 位虚拟机使用 1GB 内存、64 位虚拟机使用 1GB 内存和 64 位虚拟机使用 3GB 内存进行了性能测试，以体现增大可用内存规模所带来的性能提升效果。表 4-6 为三种情形下 SPECjvm2008 的 21 项测试数据对比。

表 4- 6 32 位与 64 位 Java 虚拟机 SPECjvm2008 测试数据对比表

测试集	32 位（ops/m）	64 位（ops/m）	vs32 性能提升(%)	64 位（3GB）（ops/m）	vs32 性能提升(%)	vs64 性能提升（%）
compiler.compiler	12.05	9.49	-21.24%	12.27	1.83%	29.29%
compiler.sunflow	4.76	3.83	-19.54%	5.3	11.34%	38.38%
compress	16.26	16.23	-0.18%	18.2	11.93%	12.14%
crypto.aes	4.59	4.44	-3.27%	5.08	10.68%	14.41%
crypto.rsa	6.47	7.91	22.26%	8.88	37.25%	12.26%
crypto.signverify	13.25	14.06	6.11%	16.05	21.13%	14.15%
derby	3.66	3.69	0.82%	4.58	25.14%	24.12%
mpegaudio	6.53	7.08	8.42%	7.84	20.06%	10.73%
scimark.fft.large	1.49	2.01	34.90%	3.01	102.01%	49.75%
scimark.lu.large	0.26	0.26	0.00%	0.42	61.54%	61.54%
scimark.sor.large	1.31	1.34	2.29%	1.95	48.85%	45.52%
scimark.sparse.large	0.82	0.85	3.66%	1.11	35.37%	30.59%
scimark.fft.small	15.48	22.3	44.06%	24.8	60.21%	11.21%
scimark.lu.small	8.28	8.46	2.17%	9.71	17.27%	14.78%
scimark.sor.small	13.81	14.4	4.27%	16.72	21.07%	16.11%
scimark.sparse.small	5.32	5.53	3.95%	7.26	36.47%	31.28%
scimark.monte_carlo	5.56	4.94	-11.15%	5.56	0.00%	12.55%
serial	4.93	4.17	-15.42%	4.6	-6.69%	10.31%
sunflow	4.95	4.88	-1.41%	6.15	24.24%	26.02%
xml.transform	10.81	9.48	-12.30%	11.2	3.61%	18.14%
xml.validation	14.53	11.54	-20.58%	13.85	-4.68%	20.02%

由表中数据可见，在 1GB 内存下，64 位 Java 虚拟机比 32 位 Java 虚拟机平均性能提升 1.14%，性能最高提升 44.06%。

在 3GB 内存下，64 位虚拟机比 1GB 内存 64 位虚拟机性能平均提升 17.62%，性能最高提升 49.75%。

在 3GB 内存下，64 位虚拟机比 32 位 1GB 内存下平均性能提升 18.97%。性能最高提升 102.01%。

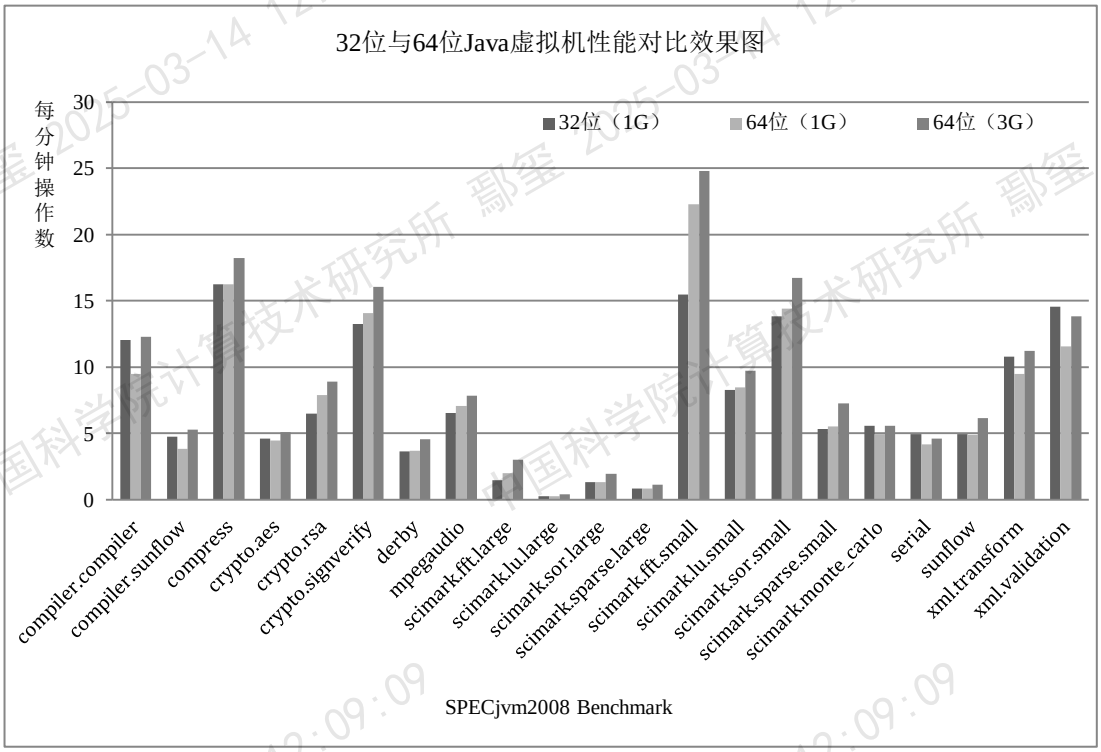


图 4- 7 32 位与 64 位 Java 虚拟机性能对比图

图 4-7 为三者的性能对比效果图。图中横坐标为 SPECjvm2008 测试集中的各个子项目，纵坐标为每分钟操作数，数值越大表示性能越好。

在某些项目上，如 compiler.compiler、derby、serial、xml，64 位虚拟机比 32 位虚拟机有轻微的下降。通过对测试程序和虚拟机运行过程行为的分析，其原因是由于在程序中存在大量的堆内存访问行为，而由于 64 位软件中存在指针膨胀，对于 64 位地址指针的加载和立即数的拼接都需要更多数目的指令，从而在这一行为上的效率要略低于 32 位虚拟机。而在实际应用程序中，内存访问的频率要远低于这几项 benchmark 测试项目。因此，整体上的实际性能不会因此而受到明显的影响。再加上 64 位 CPU 设计的初衷主要就是面向服务器计算（尤其是高性能数值计算）的，更贴近于 scimark 等数值计算项目的性能提升效果。

4.7 本章总结

本文测试了第三章的三项优化效果。分别是：查找表优化获取 Java 线程结构首地址的实现，优化后性能最高提升 166.67%。平均性能提升 13.7%。寄存器缓存优化 Java 线程结构首地址的实现，优化后性能最高可提升 22.6%。平均性能提升 0.33%。JNI 接口调用优化后，性能最高有 59.5%的提升。平均性能提升幅度为 9.1%。safepoint 优化后性能最高提升 11.8%。平均性能提升 2.26%。接着测试了 32 位虚拟机在 1GB 内存、64 位虚拟机在 1GB 内存和 3GB 内存下的性能效果。经测试数据表示，64 位虚拟机在 3GB 内存下性能比 32 位虚拟机平均提升 18.97%。充分验证了 64 位虚拟机增大可用内存规模所带来的性能提升。

本文实现的 64 位 OpenJDK 6 虚拟机，已被成功的集成到龙芯的实际应用产品中。可以很好的支持多种中间件以及第三方软件如永中 Office、金蝶中间件、东方通中间件等；很多系统软件如工信部门户网站系统、天津数字档案系统、广东信息厅 OA 系统等都将本文实现的 JDK 作为应用开发工具。此外，目前龙芯公司使用的邮件服务器 Web 前端就是运行在 MIPS 64 位 OpenJDK 6 上的。

中国科学院计算技术研究所 鄢銮 2025-03-14 12:09:09

第五章 总结与展望

5.1 本文工作总结

本小节从以下三方面总结本文的工作内容：

第一，本文首先分析了移植 MIPS 64 位 Java 虚拟机的迫切性与必要性，主要表现在：1、目前，大多数 Java 应用程序对于内存的需求越来越大，而 32 位虚拟机只提供不到 1GB 的内存，这大大限制了软件的性能和并发访问需求。2、64 位国产高性能处理器的成熟研制要求与之配套的软件环境支撑来发挥整体性能。3、64 位本身具有高运算能力，高精度，多负载，多并发的优势。接着通过阅读大量虚拟机相关的国内外文献，对虚拟机进行模块化研究与分析，为后续的移植和优化做好了充分的准备。

第二，本文完成了 OpenJDK 6 在 MIPS 64 位上的整体移植，目前虚拟机运行非常稳定，已成功集成到多家客户产品中。如中标麒麟系统、中科方德操作系统等。可以很好的支持多种中间件，如东方通中间件，金蝶中间件等，被广泛应用于多个政府系统中。如，工信部门户网站系统，广东信息厅 OA 系统等。

第三，本文移植完成后，在与 32 位 Java 虚拟机进行了性能对比分析的基础上，提出了三种优化思路。1、本文采用查找表的方法对 get_thread 方法的实现进行了优化，经过优化后，性能最高提升 166.61%，平均提升 13.7%，接着采用寄存器缓冲的方式对其进行进一步的优化，优化后，性能最高可提升性能 12.9%。2、本文针对原有只能通过解释执行调用 native 方法的局限，提出了一种通过编译执行调用 native 方法的优化实现，经过优化后，性能最高提升 59.5%，平均提升 9.1%。3、本文针对 safepoint 页面分配进行了低 32 位地址空间分配优化，降低访问 safepoint 页面地址的指令条数，经实验结果表明，性能最高可提升 11.8%。

5.2 下一步工作和研究方向

接下来的研究工作从以下两方面展开：

第一，本文只实现了 64 位客户版编译器，其优化力度仍比较粗糙，针对未来大量服务器应用程序对于峰值性能和并发吞吐的需求，接下来首要任务就是完成 64 位服务器版编译器的移植，服务器版编译器是针对本文移植的客户版编译器而言的，具有更强的优化力度和更高的峰值性能。

第二，本文只针对 Java 虚拟机中的局部功能进行了初步优化，尚未对 Java

虚拟机进行完全的优化，后续还存在能够继续挖掘以达到性能跨幅度提升的工作。例如，可以对垃圾回收算法进行串行到并行的改造，充分利用多核 CPU 资源，提升垃圾回收性能。

中国科学院计算技术研究所 鄢銮 2025-03-14 12:09:09

参考文献

[1] Marc Fleury, Francisco Reverbel. The JBoss extensible server[C]. Proceedings of the ACM/IFIP/USENIX 2003 International Conference on Middleware, June 2003.

[2] 中国科学院计算技术研究所，北京龙芯中科技术有限公司.龙芯GS464处理核手册[S]，2010.

[3] James Gosling, Bill Joy, Guy L. Steele, Java Language Specification[S], Addison-Wesley Longman Publishing Co., Inc. Boston, MA, 1996.

[4] Tim Lindholm, Frank Yellin, Java Virtual Machine Specification[S], Addison-Wesley Longman Publishing Co., Inc. Boston, MA, 1999.

[5] S. Liang and G. Bracha. Dynamic class loading in the Java™ virtual machine[C]. In Proc. Conf. on Object-Oriented Programming, Systems, Lanages, and Applications. ACM Press, 1988, pp. 36-44.

[6] Goldberg, A. A specification of Java loading and bytecode verification[C]. In ACM Conference on Computer and Communications Security, 1998. pp. 49–58.

[7] Sun Microsystems, Memory Management in the Java HotSpot Virtual Machine, April 2006.

[8] P. Cheng and G. Blelloch. A parallel, real-time garbage collector[C]. Proceedings of ACM SIGPLAN Conference on Programming Languages Designed Implementation, June 2001. pp. 125-130.

[9] BJones, R. Lins, R. Garbage Collection: Algorithms for Automatic Dynamic Memory Management[C]. New York, 1996.

[10] David F. Bacon, Ravi Konuru, Chet Murthy, Mauricio Serrano. Thin Locks: Featherweight Synchronization for Java[C]. Proceedings of the SIGPLAN’98 Conference on Proguamming Language Design and Implementation, 1998.

[11] Russell, K., Detlefs, D. Eliminating synchronization-related atomic operations with biased locking and bulk rebiasing[C]. Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications. ACM Press, New York, 2006.

[12] Takeshi Ogasawara, Hideaki Komatsu, Toshio Nakatani. A Study of Exception Handling and Its Dynamic Optimization in Java[C]. Proceedings of the 16th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications, November 2001.

[13] MCCARTHY, J. Recursive functions of sym-bolic expressions and their computation by

machine[C]. Part I.Commun. ACM 3, 4, 1960. pp. 184-195.

[14] John Aycock. A Brief History of Just-In-Time[C]. ACM Computing Surveys, June 2003. pp. 97-113.

[15] Timothy Cramer, Richard Fridman, Terrence Miller, David Seberger, Robert Wilson, Mario Wolczko. Compiling Java Just In Time[C]. Sun Microsystems, Inc.

[16] 杨勇强. 基于四核龙芯3号平台的HotSpot6移植与优化[D]. 北京：中科院计算所，2012.

[17] 蔡嵩松. 软硬件协同设计的Java虚拟机系统实现与性能优化[D]. 北京：中科院计算所，2011.

[18] Azul VM [EB/OL]. http://www.azulsystems.com/products/vega/AVM.

[19] Liquid VM [EB/OL]. http://www.theserverside.com/news/thread.tss?thread_id=43424.

[20] Harmony VM [EB/OL]. http://harmony.apache.org/subcomponents/drlvm/.

[21] Dalvik VM [EB/OL]. http://en.wikipedia.org/wiki/Dalvik_(software).

[22] JRockit VM [EB/OL]. http://www.oracle.com/technetwork/middleware/jrockit/overview/index.html.

[23] J9 VM [EB/OL]. http://www.ibm.com/developerworks/cn/ibmi/library/i-cn-j9vm/.

[24] Sun Microsystems, Java HotSpot Virtual Machine, A Technical White Paper.

[25] OpenJDK 6 [EB/OL]. http://openjdk.java.net/projects/jdk6/.

[26] Ron Cytron, Jeane Ferrante, Barry K. Rosen, and Mark N. Wegman: Efficiently Computing Static Single Assignment Form and the Control Dependence Graph[C]. ACM TOPLAS, October 1991. pp. 451-480.

[27] Danilo Bruschi, Lorenzo Martignoni, and Mattia Monga. Dectecting Self-mutating Malware Using Control-Flow Graph Matching[C]. Springer-Verlag Berlin Heidelberg, 2006. pp. 130-142.

[28] Wurthinger, T., Wimmer, C., Mossenbock, H. Array bounds check elimination for the Java HotSpot™ client compiler[C]. Proceedings of the International Conference on Principles and Practice of Programming in Java. ACM Press, New York, 2007.

[29] Traub, O., Holloway, G., Smith, M. D. Quality and speed in linear-scan register allocation[C]. Proceedings of the ACM SIGPLAN Conference. New York, 1998. pp. 142-150.

[30] (英)Dominic Sweetman 著，李鹏，鲍峥，石洋等译，MIPS 体系结构透视[M]，机械工业出版社，2008.

[31] Thomas Kotzmann, Christian wimmer, Hanspeter Mossenbock, Thomas Rodriguez, Kenneth Russell, David Cox. Design of the Java HotSpot™ Client Compiler for Java 6[C]. ACM Transactions on Architecture and Code Optimization, Vol.5, May 2008.

[32] Michael Paleczny, Christopher Vick, Cliff Click. The Java HotSpot™ Server Compiler[C]. Proceedings of the 2001 Symposium on Java™ Virtual Machine Research and Technology Symposium, April 2001.

[33] Adl-Tabatabai, A.-R., Cierniak, M., Lueh, G.-Y., Parikh, V. M., Stichnoth, J. M. Fast, effective code generation in a just-in-time Java compiler[C]. Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation. ACM Press, New York, 1998.

[34] Sheng Liang. The Java Native Interface[S]. Programmer’s Guide and Specification.

[35] Urs Holzle, Craig Chambers, David Ungar. Optimizing Dynamically-Typed Object-Oriented Languages with Polymorphic Inline Caches[C]. In: ECOOP'91 European Conference on Object-Oriented Programming, vol. 512, Lecture Notes in Computer Science, P. America, Ed. Springer Berlin / Heidelberg, 1991. pp. 21-38.

[36] D. Detlefs, O. Agesen. Inlining of virtual methods[C]. In 13th European Conference on Object-Oriented Programming, June 1999.

[37] T. Suganuma, T. Yasue, M. Kawahito, H. Komatsu, T. Nakatani. A dynamic optimization framework for a Java just-in-time compiler[C]. In ACM Coference, Oct 2001. pp. 185-190.

[38] Fink, S.J, Qian, F. Design, implementation and evaluation of adaptive recompilation with on-stack replacement[C]. Preceeding of the International Symposium on Code Generation and Optimization. IEEE Computer Society, CA, 2003. pp. 241-245.

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

致 谢

本文能够顺利完成首先得感谢我的导师张戈老师在论文期间给我的指导。在整个研究生期间，导师给了我很多的关心和帮助，当我遇到困难不得其解时，导师总会积极地引导我思考，指导我思路，他现在是龙芯通用事业部总经理，经常能在龙芯的重要会议上听到老师的发言，他的处事方式及思维都深深的影响了我，是我在今后的工作岗位中学习的榜样。

感谢王剑老师和章隆兵老师在研究生期间给予我科研和学习上的指导与帮助。王老师对工作的认真负责，对科研的耐心热忱，对学生的和蔼可亲都深深地印入我的脑海，章老师对学生的严格要求，对学术的严谨是我学习的好榜样。

感谢靳国杰老师在这段时间给我的指导和帮助，他每天必写工作日志的工作习惯，对待问题的逻辑思维方式，对工程的热爱，对学术的执着等等这些让我敬佩不已的同时也受益匪浅。

感谢蔡嵩松师兄在我研究生期间给予的关心与指导，当我遇到难题，遇到困难时，只要跟我师兄请教，他都会很耐心地跟我讲解。在整个研究生期间，师兄就像我的良师益友，给了我很多支持和帮助。

感谢刘奇师兄，敖琪师兄，杨勇强师兄，感谢所有龙芯通用 Java 组的人，是你们一直支持我，鼓励我，研究生阶段能和你们在一起，我感到非常幸运。

感谢我的舍友兼师姐，在写论文期间，她给了我很多鼓励及建议，感谢研究生期间的所有给与我帮助的人。

感谢我的家人和朋友，在这些年对我的支持和关爱。再多的话都无法表达我的感激之情。

最后再次衷心的感谢所有关心和帮助我的人，祝你们一切都好！

武 慧

2013 年 5 月

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09

作者简介及在学期间发表的学术论文与研究成果

作者简历：

1987 年 12 月 08 日出生于山西太原。

2005 年 09 月——2009 年 06 月，在湘潭大学信息工程学院获得学士学位。

2010 年 09 月——2013 年 06 月，在中国科学院计算技术研究所硕士学位。

工作经历：

2009 年 06 月——2009 年 09 月，在北京长信通信息技术有限公司工作。

已发表（或正式接受）的学术论文：

参加的研究项目及获奖情况：

[1] 国家“核高基”科技重大专项课题，高性能多核 CPU 研发与应用，2010-2012

[2] 国家“核高基”科技重大专项课题，龙芯安全适用计算机 CPU 研发与应用， 2010-2012

中国科学院计算技术研究所 鄢玺 2025-03-14 12:09:09